ChatGPT

Zero-to-One 3D 웹 인테리어 플랫폼 개발 가이드

1. 기존 프로젝트 UI 품질 이슈 분석

현재 구현된 코드와 시스템 아키텍처를 검토한 결과, 시각적으로 품질이 낮아 보이는 원인은 **그래픽 처리의 단순함**과 **카메라 제어 미흡**에서 비롯되었습니다. 우선 **쉐이더 미적용** 문제로, 장면에 커스텀 쉐이더나 고급 머티리얼이 사용되지 않고 기본적인 Three.js 표준 머티리얼만 사용되어 질감과 조명이 단조롭습니다 [1]. 예를 들어 벽체는 **BoxGeometry**로 생성하고 색상만 지정된 **MeshStandardMaterial**로 렌더링되어 표면 질감이나 반사가 거의 없는 상태입니다 [1]. 또한 **단순 지오메트리 구성**으로 인해 모델 디테일이 부족합니다. 벽과 바닥/천장 등이 모두 직육면체나 평면으로만 구현되어 있어 입체감이 떨어집니다 [1] [2]. 조명도 주변광(AmbientLight) + 단일 방향광(DirectionalLight)의 최소 구성으로, 글로벌 조도나 그림자는 일부 적용되었지만 환경 맵이나 후처리 효과가 없어 현실감이 낮습니다 [3].

게다가 **시점(viewpoint) 처리의 미흡**으로 UI 사용성이 떨어집니다. 1인칭(FPS) 시점 제어가 완전히 구현되지 않아 WASD 이동이나 마우스 룩 기능이 제한되었고, 1인칭/탑뷰 전환도 부자연스러운 것으로 보입니다 [4] [5]. 실제 코드에서도 PointerLockControls를 도입하려 했으나 완전히 동작하지 않아, **walk 모드에서 WASD 이동이 안 되는 문제**가 보고되었습니다 [4]. 이러한 카메라 제어 미흡은 사용자가 공간을 탐색할 때 답답함을 느끼게 하고, 시점 전환 시 어색한 이동이나 튕김 현상이 있었을 가능성이 높습니다.

UI 품질 저하는 새로운 기능 추가 없이 시각적인 부분만 바뀌었기 때문인데, 예를 들어 기존 Gemini 기반 도면 분석이 실패하여 3D 씬에 매우 작은 6×4m 박스 하나만 나타나는 등 결과물이 부실했습니다 [4]. 기능적으로는 큰 변화가 없었음에도 불구하고, 잘못된 도면 해석 출력으로 엉뚱한 장면이 렌더링되고 성능까지 저하되면서 사용자 체감 품질이 악화된 것입니다. 실제 로그에서도 Gemini 모델의 실패로 **더미 데이터**만 반환되어 작은 박스형 벽체가 생성되었고, 원하는 도면을 넣었지만 제대로 반영되지 않았음을 확인할 수 있습니다 [6]. 요약하면, **부족한 그래픽 표현(조명·재질·모델 디테일)**과 **미완성된 시점 제어**가 겹쳐 **"UI 품질이 나빠졌는데 기능 향상은 없다"**는 인상을 주게 되었습니다.

2. 기존 코드 리팩토링 vs 신규 구축 접근 비교

이러한 문제를 해결하기 위해 **기존 프로젝트를 리팩토링할지**, 아니면 **새로운 디렉토리에서 제로베이스(Zero-to-One)로 재구축할지** 결정해야 합니다. 각 접근의 **효율성**과 **기술 확장성**을 비교하면 다음과 같습니다.

- **기존 프로젝트 리팩토링**: 현재 코드베이스의 뼈대(React+Three.js 구조, Supabase 연동 등)를 유지하면서 문제점만 개선하는 방법입니다. 장점은 이미 구현된 기능(예: 프로젝트 생성/저장, 기본 3D 편집 등) [7] 을 활용할 수 있어 개발 시간이 절약됩니다. 특히 Supabase 인증/DB 연동, Zustand 상태관리 등은 정상 동작 중이므로 이를 재사용 가능합니다. 따라서 부분적인 리팩토링으로 **WallManager**의 메모리 누수 제거와 **조명 최적화**를 진행하고 [8], **FurnitureMesh** 컴포넌트에 `React.memo`를 적용하는 등 성능 개선을 도모할 수 있습니다 [8]. 단, 리팩토링으로는 기존 구조상의 한계를 완전히 극복하기 어려울 수 있습니다. 예를 들어, 현 아키텍처에서는 1인칭/탑뷰 모드 전환이나 멀티유저 동기화 같은 신규 기능을 끼워 넣기엔 구조적인 수정이 많이 필요할 수 있습니다. 기존 코드에 이미 기술부채(technical debt)가 쌓여 FPS가 30~40에 머무는 등 최적화 한계도 나타났습니다 [9]. **요약**: 리팩토링은 **개발 속도** 면에서 유리하나, 기존 구조 제약으로 **확장성** 확보에는 한계가 있습니다.

- **신규 구축 (Zero-to-One)**: 새로운 폴더/프로젝트에서 처음부터 이상적인 구조로 개발을 시작하는 방식입니다. 이 접근은 **처음부터 정교한 아키텍처 설계**를 적용해 요구되는 기능들을 체계적으로 구현할 수 있습니다. 예를 들어, 시스템 아키텍처 다이어그램, 디렉토리 구조, 데이터 흐름 (인증, 프로젝트 생성, 3D 편집, 실시간 협업) 등을 사전에 정의하여 개발할 수 있습니다 [10]. 이미 내부 문서에서도 `architecture.md`로 폴더 구조와 `supabase-setup.md`로 백엔드 설정을 완전히 재작성하는 등 신규 설계 움직임이 있었습니다 [11] [12].

Zero-to-One 접근의 장점은 **기술 확장성**이 뛰어나다는 점입니다. React Three Fiber(R3F) 기반의 최신 생태계 도구들을 처음부터 통합하고, 멀티뷰 모드나 멀티유저를 고려한 상태관리(Zustand)와 실시간 통신 구조를 깨끗하게 설계할 수 있습니다. 또한 성능과 보안을 최우선으로 고려하여, 불필요한 패키지나 구 코드를 배제하고 최적화된 번들링, WebGL/WebGPU 대응을 내재화할 수 있습니다. 단, 처음부터 구축하기 때문에 **초기 개발 공수**가 커질 수 있고, 기존 코드에서 가져올 수 있는 부분도 다시 작성해야 하는 비효율이 있습니다. 하지만 현재 남은 구현 과제가 명확하고 (예: 도면→3D 변환, 1인칭 모드, 랜딩 페이지 개선 등 [13]) 대부분 신규 개발이 필요한 부분이므로, 애매하게 리팩토링을 이어가는 것보다 **처음부터 제대로 만드는 편이 오히려 효율적**일 수 있습니다.

어떤 접근이 적절한가? 현재 프로젝트의 핵심 약점(그래픽 퀄리티, 정확한 도면 인식, 부가 기능 부족)을 해결하려면 **신규 아키텍처 설계**가 바람직합니다. 이미 60% 정도는 완성된 상태에서 남은 40%를 3~4일 내 마무리할 수 있다는 내부 분석이 있지만 [7], 이는 어디까지나 임시 방편일 수 있습니다. 요구사항을 **100점짜리 플랫폼** 수준으로 끌어올리기 위해서는, 기존 한계를 답습하기보다 **근본적인 구조 개선**이 필요합니다. 내부 논의에서도 "Plan2Space 100점 달성 가이드"를 마련하며 새로운 파일들과 문서를 구성한 것으로 보아 [14], **Zero-to-One 재구축 방향**이 이미 제시되었습니다. 요약하면, **현재 프로젝트는 프로토타입 단계**로서 시각적 완성도와 구조적 완전성이 부족하므로, **확장성과 품질을 최우선 고려한 새 구조로의 재구축이 더 적절**합니다. 다만, 기존 구현에서 검증된 모듈(예: Supabase 연동, 일부 유틸 함수 등)은 선별적으로 가져와 활용하여 개발 효율을 높이는 하이브리드 전략을 취할 수 있습니다.

3. 고품질 3D 웹 인터페이스 구현 전략 (Dverso Studio/Bruno Simon 스타일)

높은 시각적 완성도의 3D 웹 UI를 구현하려면 **React Three Fiber(R3F)** 생태계의 다양한 고급 라이브러리를 적극 활용하고, **WebGL 기반 커스텀 쉐이더와 조명 기법**을 조합해야 합니다. Bruno Simon의 3D 포트폴리오나 Dverso Studio의 인터랙티브 사이트처럼 부드럽고 몰입감있는 3D 웹 경험을 만들기 위한 전략을 정리하면 다음과 같습니다.

- **R3F 및 Drei 활용**: React Three Fiber는 Three.js를 React 방식으로 사용하도록 해주므로, 여기에 **@react-three/drei**와 같은 보조 라이브러리를 추가하면 개발 생산성과 결과물 품질이 향상됩니다. Drei는 Bruno Simon 같은 선도적인 3D 웹 개발자들이 공동으로 관리하는 유틸 모음으로, 카메라/컨트롤, 환경맵, 모델 로더 등 유용한 컴포넌트를 제공합니다. 예를 들어 **OrbitControls** (드래그로 3D 회전/이동 제어)와 **PointerLockControls** (클릭으로 1인칭 고정 시점 제어) 등을 바로 사용할 수 있고 [15], **useGLTF** 훅을 통해 복잡한 3D 모델(GLTF/GLB)을 편리하게 로드할 수 있습니다 [16]. Bruno Simon 스타일의 사이트에서는 이러한 컨트롤로 자연스러운 카메라 움직임을 구현하고, 여러 GLTF 자산(자동차, 건물 등)을 쉽게 배치했습니다. 우리 프로젝트에서도 Drei의 `<Environment>` 컴포넌트를 활용해 HDRI 환경맵을 설정하거나, `<ContactShadows>`로 바닥 접촉 그림자를 추가하는 등 **시각적 리얼리티**를 높일 수 있습니다. **@react-three/drei**에 포함된 다양한 프리미티브 (Box, Cylinder, Sphere 등)와 데코레이터(Text, Html, Billboard 등)를 활용하면 저수준 Three.js 코드 작성 없이도 풍부한 3D UI 요소를 구성할 수 있습니다 [17].

- **후처리 효과(Postprocessing)**: 고품질 3D의 완성도를 높이려면 **후처리(Post-processing)**를 적용하여 화면 효과를 세련되게 다듬어야 합니다. R3F에서는 `@react-three/postprocessing`이나 Drei의 `<Effects>`를 통해 Three.js EffectComposer 기반 후처리 효과들을 손쉽게 붙일 수 있습니다. 주요 효과로 **블룸(Bloom)** 효과를 적용하면 밝은 영역의 빛 번짐을 표현해 현실감을 높이고, **안티앨리어싱 개선(SSAA/SMAA)**으로 도형의 계단 현상을 줄일 수 있습니다. 또한 **피사계 심도(Depth of Field)** 효과를 주어 먼/가까운 객체의 포커스를 조절하면 카메라 전환 시 영화 같은 느낌을 줄 수 있습니다. Bruno Simon 사이트에서도 자동차 헤드라이트의 빛 번짐이나, 씬 전환 시의 포커스 효과 등으로 몰입감을 준 바 있습니다. 이외에 **색보정/필터**를 걸어 특정 분위기의 룩앤필을 만들 수도 있습니다 (예: 콘트라스트 조정, 채도 강조 등으로 산뜻한 느낌 연출). 후처리를 과용하면 성능이 떨어질 수 있으므로 필요한 효과만 선택하여, PC/모바일 플랫폼별로 활성화 수준을 다르게 하는 최적화도 고려해야 합니다.

- **Maath 및 기타 유틸**: **@react-three/maath** 라이브러리는 3D 수학과 애니메이션 보간 등에 유용한 도구를 제공합니다. 예를 들어 `maath`의 `damp()` 함수들을 이용하면 카메라 이동이나 오브젝트 이동을 부드럽게 보간(damping)할 수 있습니다. 사용자 입력이 갑자기 변해도 움직임을 서서히 따라가도록 하여 **부드러운 인터랙션**을 구현하는 데 도움이 됩니다. 또한 벡터/쿼터니언 연산, 충돌 계산 등의 수학 처리를 고성능으로 수행하는 도구도 포함되어 있습니다. 이와 함께 Three.js 자체 유틸 (BufferGeometryUtils 등)을 활용해 반복되는 지오메트리를 병합하거나, **Instancing**을 통해 다수의 오브젝트를 효율적으로 그리는 것도 중요합니다. Dverso 등의 사례를 보면, 다량의 3D 오브젝트를 화면에 배치하면서도 성능을 확보하기 위해 instanced mesh 기법과 모듈화된 수학 계산을 사용합니다.

- **Framer Motion과 UI 통합**: 3D 씬과 웹 UI 요소를 자연스럽게 연계하려면 **Framer Motion** 같은 애니메이션 라이브러리를 활용할 수 있습니다. Framer Motion은 React용 애니메이션 패키지로, CSS 기반 UI 패널이나 버튼의 등장/퇴장, 상태 변화 애니메이션을 손쉽게 구현하게 해줍니다. 예를 들어, 1인칭 모드 진입 시 화면에 크로스헤어나 인벤토리 바를 페이드인/아웃하거나, 탑뷰 모드에서 사이드바 패널이 부드럽게 슬라이드되는 효과를 줄 수 있습니다. 또한 Framer Motion과 Three.js를 조합하면 카메라 전환 애니메이션도 가능한데, R3F의 `<useFrame>` 훅과 조합하여 카메라 포지션을 Framer Motion의 값으로 lerp하면 일정 시간에 걸쳐 부드럽게 시점을 이동시킬 수 있습니다. UI와 3D객체가 겹치는 경우 Drei의 `<Html>` 컴포넌트로 3D 공간에 DOM요소를 띄울 수 있는데, 이때 Framer Motion을 적용해 부드러운 팝업 효과를 주는 것도 고려해볼 만 합니다. 예를 들어 가구에 마우스를 올리면 해당 가구 이름을 `<Html>`로 띄우되, Framer Motion으로 scale-up 되는 애니메이션을 주어 강조하는 방식입니다. 전체적으로 **3D 요소의 움직임 + 2D UI의 트랜지션**을 일관성 있게 디자인하면, 사용자 입장에서 매끄럽고 현대적인 인터페이스를 경험하게 됩니다.

4. WebGL/WebGPU 기반 쉐이더 및 조명 최적화 방안

쉐이더(Shader) 기술을 적극 도입하여 그래픽 퀄리티를 향상시키고, **베이크드 조명**과 **실시간 조명**을 효율적으로 조합하는 전략이 필요합니다. WebGL 환경 하에서 Three.js는 기본적으로 GLSL 쉐이더를 사용하지만, 향후 WebGPU로 전환될 경우를 대비한 설계도 염두에 둡니다.

- **맞춤형 쉐이더 활용**: 현재는 MeshStandardMaterial 등의 내장 머티리얼만 사용 중이지만, 고유한 비주얼을 위해 **커스텀 쉐이더**를 일부 도입하는 것이 좋습니다. 예를 들어 벽지나 바닥재의 질감을 살리기 위해 **노말 맵**이나 **패럴랙스 맵**을 사용하는 쉐이더를 적용할 수 있습니다. Drei의 `<shaderMaterial>` 훅을 이용하면 GLSL 코드를 직접 삽입한 커스텀 머티리얼을 React 컴포넌트로 만들 수 있습니다. Bruno Simon 스타일의 연출을 위해 만약 카툰 렌더링을 원한다면, **토룩 쉐이딩(toon shading)** 쉐이더를 적용해 경계가 뚜렷한 명암을 표현할 수 있고, 오브젝트 테두리에 **아웃라인** 쉐이더를 후처리로 넣어 만화같은 표현을 할 수도 있습니다. 반대로 사실적인 표현을 원하면 PBR 매터리얼에 **환경 반사**와 **조명 계산**을 커스터마이즈하여 광택이나 투명 효과를 극대화할 수 있습니다. WebGL 기반으로 작성한 GLSL은 추후 Three.js가 WebGPU를 지원하게 되면 WGSL로의 전환 작업이 필요할 수 있지만, Three.js 레벨에서 NodeMaterial 등을 쓰면 어느 정도 호환성을 유지하면서 노드 기반 쉐이더를 구성할 수 있습니다. 요약하면, **핵심 오브젝트에 한해 커스텀 쉐이더를 적용**하여 시각적 포인트를 줌으로써 전체 완성도를 끌어올리고, 나머지는 성능을 위해 표준쉐이더(혹은 Nodes)를 사용하는 균형 잡힌 접근이 필요합니다.

- **베이크드 라이팅(Baked Lighting)**: 실시간 조명의 한계를 보완하기 위해 **광원 효과를 미리 굽는** 기법을 도입합니다. 실내 인테리어의 전역 조명(Global Illumination)은 계산이 무겁기 때문에, Blender 등의 툴을 활용해 **라이트맵(Lightmap)**을 베이크한 후 Three.js에 가져오는 방법이 있습니다. 예를 들어 벽과 바닥에 대해 앰비언트 오클루전이나 간접조명 효과를 라이트맵 텍스처로 구워두고, Three.js 머티리얼의 `lightMap` 슬롯에 적용하면 실시간 광원이 없어도 부드러운 음영이 나타납니다. Polyhaven 등에서 제공하는 HDR 환경맵으로 **IBL(Image-Based Lighting)**을 사용하면서, 추가로 라이트맵을 곱산하면 공간 구석진 곳의 어둠 표현이나 빛 퍼짐을 리얼하게 표현할 수 있습니다. 베이크시 해상도가 높아지면 텍스처 용량 문제가 있으므로 방크기 별로 적절한 해상도를 선택하고, 모바일용으로는 해상도를 낮춘 버전을 쓰는 등 조정이 필요합니다.

- **실시간 조명 최적화**: 베이크된 조명이 있다고 해도, 움직이는 객체나 시간 변화에 따른 빛 표현을 위해 어느 정도 **실시간 조명**은 필요합니다. 여기서는 **효율적인 광원 구성**이 중요합니다. 제안하는 조합은 다음과 같습니다:

- **HemisphereLight**: Three.js의 HemisphereLight는 하늘색과 지면색 두 색상의 반구 조명으로, 특별히 그림자를 생성하지 않으면서 전역적인 앰비언트 라이트를 줄 수 있습니다. 실내라 해도 창문을 통한 하늘빛, 바닥 반사빛 등의 효과를 이 조명 하나로 간단히 표현할 수 있습니다. HemisphereLight를 약하게 깔아두면 씬 전체의 암부(detail in shadows)를 살릴 수 있습니다.
- **DirectionaLight (태양/주광)**: 해가 들어오는 창문이 있거나, 또는 주된 광원 역할로 한두 개의 DirectionalLight를 사용합니다. 이 광원에는 **그림자(shadow)**를 활성화하여 실시간으로 물체 그림자를 생성하도록 합니다 [18] [19]. 다만 DirectionalLight의 그림자 맵 크기를 적절히 조절하고 [20], `shadow.camera` 범위를 필요 최소한으로 설정하여 퍼포먼스를 관리해야 합니다. 예컨대 방 전체를 커버할 만큼만 그림자를 계산하게 하고, 그림자 화질은 PC에선 2048px 이상, 모바일에선 1024px 정도로 가변 적용합니다.
- **Point/Spot Light (국부 광원)**: 전등이나 스탠드 등의 효과를 위해 PointLight나 SpotLight를 배치할 수 있습니다. 그러나 다수를 사용하면 성능 저하가 크므로, 진짜 필요한 핵심 조명만 켜고 나머지는 분위기만 살리는 **듀미 에미시브 재질**로 대체할 수 있습니다. 예를 들어 램프 모형의 전구 부분은 자체 발광(Emissive) 재질로 만들어 실제 광원을 켜지 않고도 빛나는 느낌을 주거나, 또는 SpotLight를 사용하되 그림자는 끄는 식입니다.

- **실시간 AO와 기타 효과**: 베이크를 못 했을 경우를 대비해, Screen-Space Ambient Occlusion(SSAO) 같은 후처리 이펙트를 사용하면 즉각적인 접촉음영 효과를 낼 수 있습니다. 또한 Bloom 효과로 빛번짐을 표현하면 광원의 밝기 체감이 올라갑니다. 이 모두를 **적절히 조합**하여, 정적인 전역빛은 베이크로 처리하고 움직이는 그림자나 눈에 띄는 하이라이트만 실시간으로 처리하는 구성으로 최적화합니다.

- **WebGL vs WebGPU 준비**: 현재 Three.js는 WebGL 기반으로 안정적이지만, 머지않아 WebGPU를 활용한 렌더링이 주류가 될 수 있습니다. WebGPU는 GPU 성능을 더 끌어낼 수 있지만 초기 단계이므로, 당장 WebGPU 전용으로 개발하진 않더라도 **Shader 코드를 모듈화**하고 **Three.js의 WebGPURenderer 옵션**에 실험적으로 대응해보는 것은 의미가 있습니다. 예를 들어 Three.js에서 제공하는 `WebGPURenderer`를 조건부로 사용해보고, 쉐이더를 작성할 때 Three.js의 `NodeMaterial`이나 `onBeforeCompile` 훅을 써두면 WebGPU 환경에서 호환성을 높일 수 있습니다. WebGPU에서는 그림자 계산 방식이나 포스트프로세싱이 약간 다르므로, 해당 부분을 추상화할 수 있는 구조(예: `useRenderer` 훅을 만들어 WebGL/WebGPU를 내부적으로 선택)로 설계할 수도 있습니다. 요약하면, **현재는 WebGL 최적화에 집중**하되 **향후 WebGPU 전환 시 최소한의 수정**으로 대응할 수 있도록 코드를 정리해두는 것이 좋습니다.

以上의 전략을 종합하면: R3F+Drei 등 **고급 생태계 도구**를 활용해 개발 효율과 기본 품질을 높이고, 필요한 경우 **커스텀 쉐이더와 후처리**로 개성있는 표현을 더합니다. 그리고 **베이크된 조명**으로 전반적인 현실감을 잡고, **실시간 조명**은 최소한으로 사용하여 성능과 퀄리티를 양립시키는 방향으로 구현합니다. 이렇게 하면 Dverso나 Bruno Simon처럼 **시각적 완성도가 뛰어나면서도 부드럽게 동작하는 3D 웹 인터페이스**를 구축할 수 있을 것입니다.

5. 도면 인식 정확도 향상 방안: 최신 Vision LLM vs 전용 AI 모델

현재 도면(건축 평면도) 이미지에서 공간 구조를 인식하는 데 사용 중인 **Gemini 모델**의 한계를 분석하고, 이를 대체하거나 보완할 수 있는 최신 **비전 언어 모델(vision LLM)** 및 **컴퓨터 비전 전문 모델**을 비교해보겠습니다. 목표는 도면에서 **방 구조, 벽체(두께/높이), 창문·문의 위치, 입구 포인트** 등을 높은 정확도로 추출하는 것입니다.

- **Gemini 모델 이슈**: 기존에 사용된 Google Gemini 기반 도면 분석은 정확도가 낮았습니다. 실제 결과를 보면 **Gemini가 도면을 제대로 해석하지 못해 더미 데이터만 반환**하고, 그 결과 3D 씬에는 6×4m 크기의 작은 상자 하나가 표시되는 수준이었습니다 [4]. 응답 파싱도 실패하여 결국 `mockResult()` 더미 값을 사용한 것이 로그로 확인되었습니다 [6]. 이러한 실패 원인은 Gemini 모델이 건축 도면 같은 도메인 특화 이미지를 이해하는 능력이 부족하거나, 프롬프트 지시를 엄격히 따르지 못했기 때문으로 보입니다. Gemini는 멀티모달 거대 모

델로 기대되었으나, 실제 활용에서는 JSON 포맷으로 복잡한 도면 정보를 내보내도록 제어하기 어려웠던 것입니다.

- **Claude 3.5 (Sonnet) 및 GPT-4 (Vision) 비교**: 최신 멀티모달 LLM으로 **Anthropic Claude 3.5** (코드명 Sonnet)나 **OpenAI GPT-4 (Vision)**를 고려할 수 있습니다. 문헌에 따르면 Claude 2 계열의 모델도 이미지 입력에 대한 실험적 기능이 추가되고 있으며, GPT-4 Vision은 이미 이미지 인식 능력을 시연한 바 있습니다. 이들 모델은 **지시 준수 능력**이 Gemini보다 뛰어나 JSON과 같은 구조화 출력 요구를 정확히 따를 가능성이 큽니다 [21]. 예컨대 Claude API를 통해 "이 도면 이미지를 분석해 방/벽/창문 정보를 JSON으로 반환하라"는 프롬프트를 주면, 응답을 JSON으로 깔끔히 내놓을 확률이 높습니다. 실제로 내부 계획에서도 Gemini 대신 **Claude Vision API로 마이그레이션**하여 정확도를 높이자는 지시가 있었습니다 [22]. Claude는 제한된 토큰 내에서 요약에 강하고, GPT-4는 복잡한 추론에 강한 경향이 있는데, **도면 해석**은 이미지의 시각적 패턴 인식 + 도메인 지식 적용(예: 벽 두께 표준, 방 이름 인식)이 모두 필요합니다. GPT-4 Vision은 건축 도면의 기호와 레이블을 꽤 잘 이해할 수 있을 것으로 기대되며, Claude도 충분한 예시와 함께 프롬프트를 주면 유사한 성능을 낼 수 있을 것입니다. 다만 두 모델 모두 **컨텍스트 크기 제한**으로 인해 매우 복잡한 도면 (예: 수십 개 방이 있는 건물 평면)일 경우 전체 정보를 하나의 응답에 담는 데 어려움이 있을 수 있습니다. 또 **API 비용** 측면에서도 이미지 인식 가능한 GPT-4는 비용이 높게 책정될 수 있으므로, 실서비스에서 빈번히 호출하기엔 부담이 될 수 있습니다. 종합하면, **Claude 3.5**나 **GPT-4 Vision**은 Gemini 대비 **현저히 높은 정확도와 안정성**을 제공할 것으로 보이며, 특히 **JSON 출력의 일관성** 면에서 유리합니다. 실제 Claude로 전환 시도도 진행되었고, 이를 통해 도면 인식 정확도를 높이고 FPS도 향상시킨다는 목표(FPS 30→60 등)가 세워졌습니다 [23] [24].

- **OpenCV 기반 접근**: LLM 대신 전통적인 **컴퓨터 비전 파이프라인(OpenCV)**을 활용하는 방법입니다. OpenCV로는 도면 이미지를 이진화/모폴로지 처리한 후 직선 검출(Hough Transform)으로 **벽체 선분**을 감지하고, 컨투어 분석으로 **방 영역**을 추출하는 접근을 생각해볼 수 있습니다. 예를 들어 벽은 일정 두께를 가진 평행 선 쌍으로 표현되므로, 엣지 검출 후 일정 간격으로 나란한 선들을 찾으면 벽 후보를 찾을 수 있습니다. 또한 문은 벽의 끊긴 부분에 아치형 또는 특별한 기호로 표시되므로, 선이 끊긴 구간이나 특정 아이콘(문을 나타내는 ㄷ자 형태 등)을 매칭해 찾을 수 있습니다. OpenCV의 Template Matching이나 Feature Matching 기법을 활용하면 창문/문의 도면 기호를 인식하도록 할 수도 있습니다 [25]. 그러나 OpenCV 기반 접근은 **내구성이 낮고 많은 튜닝**이 필요합니다. 도면 이미지마다 해상도, 선 두께, 노이즈가 다르고, 기호 역시 표준이 제각각이라 단순 알고리즘으로 모든 경우를 커버하기 어렵습니다 [26]. Stack Overflow 등의 논의에 따르면 "완전하고 좋은 솔루션은 없으며, 결국 신경망 기반으로 하는 것이 낫다"는 의견이 많습니다 [26]. 따라서 OpenCV는 **보조 수단**으로서, 예를 들어 LLM이 대략 낸 결과를 보정하거나, LLM이 못 알아챈 디테일(정확한 치수) 추출 등에 쓰는 게 현실적입니다. 한 가지 방안은, LLM이 출력한 벽 좌표가 애매하다면 OpenCV로 이미지에서 그 좌표 주변의 픽셀을 확인해 벽 선의 끝점을 미세 조정하는 것입니다.

- **Segment Anything (SAM)**: Meta의 Segment Anything 모델은 **이미지 내 임의의 객체 영역을 분할**해주는 모델로, 건축 도면에 적용하면 여러 영역(방 면적 부분 vs 벽체 vs 배경)을 구분하는데 쓸 수 있습니다. SAM은 도메인 특화가 아니어서 바로 벽만 딱 찾아주진 않지만, 예컨대 **방 내부 빈 공간 vs 선으로 그려진 벽/문 영역**은 픽셀 집합 차원에서 구분될 수 있습니다. SAM을 사용해 도면 이미지를 픽셀 단위로 세그먼트 여러 개로 나눈 후, **후처리**로 각 세그먼트를 분류하는 방법을 생각해볼 수 있습니다. 벽체는 보통 길쭉한 굵은 영역, 방 바닥은 폐곡선으로 둘러싸인 넓은 영역, 문/창문 기호는 비교적 작은 특수 모양 등의 특성이 있으므로, 세그먼트의 면적, 형태, 위치 등을 분석하면 벽 vs 방바닥 등을 식별 가능할 것입니다. 특히 SAM의 **Point-based prompt**를 활용하면, LLM 결과의 대략적 좌표를 점으로 찍어 SAM에 그 주위 영역을 분할시키는 방식도 가능합니다. 즉, LLM이 "벽이 대략 여기 있다"고 한 지점 근처를 SAM에게 분할시키면 정확한 벽 픽셀 영역을 뽑아줄 수 있다는 것입니다. 이렇게 하면 LLM의 추상적 추론과 CV의 픽셀 정확도를 결합할 수 있습니다. 다만 SAM은 결과를 **벡터 형태**로 바로 주지 않으므로, 얻어진 마스크를 다시 Polyline으로 변환해 좌표를 추출하는 과정을 추가해야 합니다.

- **Roboflow 및 딥러닝 전용 모델**: 최상의 정확도를 원한다면, 아예 **도면 분석 특화 딥러닝 모델**을 학습시키는 것도 고려해볼 만 합니다. 실제 사례로, 2021년 연구에서는 U-Net 기반 모델로 건축 평면도의 **벽체 폴리곤 추출**을 시도한 바 있습니다 [27]. 또한 상용/오픈소스 프로젝트들도 존재하는데, 예컨대 Roboflow 블로그에서는 **층별로 주석된 504장의 도면 데이터셋**으로 세그멘테이션 모델을 훈련해 벽/문 등을 분류하는 튜토리얼을 제공하

고 있습니다 [28]. 이러한 모델은 충분한 학습만 거치면 특정 스타일의 도면에 대해 사람 수준의 정확도로 구성 요소를 픽셀 단위 분류할 수 있습니다. Roboflow 플랫폼을 활용하면 데이터 수집(예: 수백 장의 도면 이미지 업로드)부터 레이블링(벽, 문, 창 등 태깅)과 모델 훈련(백본 모델 선택 및 AutoML)까지 비교적 손쉽게 진행할 수 있습니다 [29] [30]. 완성된 모델은 REST API 또는 ONNX, TensorRT 등의 포맷으로 배포하여, 우리 앱에서 **이미지 입력→모델 추론→JSON 변환** 파이프라인에 통합할 수 있습니다. 이러한 **전용 모델 접근의 장점**은, 도면의 미세한 요소(치수선, 텍스트 라벨 등)까지도 학습을 통해 인지할 수 있고 **픽셀 단위 정확도**가 높다는 점입니다 [31] [32]. 반면 단점은 데이터를 준비하고 모델을 학습/튜닝하는 데 시간과 비용이 들며, 도면 스타일이 바뀌면 일반화가 안 될 수 있다는 점입니다. 또한 LLM처럼 고차원적인 추론(예: "이 방은 거실로 보인다" 같은 의미 파악)은 못하고, 어디까지나 시각 패턴만 인식합니다. 따라서 **혼합 접근**도 고려됩니다: 딥러닝 CV 모델로 **벽/방/창 도형을 검출**하고, LLM으로 **텍스트 정보나 의미**(방 이름, 치수 단위 등)를 해석하는 협업 방안입니다. 이렇게 하면 구조 인식 정확도를 높이면서도, 모델이 이해 못 하는 맥락은 LLM이 보완하는 형태가 됩니다.

정리하면, **가장 현실적인 대안**은 우선 **Claude 3.5 혹은 GPT-4 Vision과 같은 강력한 LLM을 활용**하여 신속히 정확도를 높이고, 동시에 **OpenCV/SAM 보조**를 통해 미세 조정이나 보정 로직을 추가하는 것입니다. 그리고 장기적으로 사용자 도면 데이터가 축적된다면, **전용 딥러닝 모델**을 학습시켜 LLM을 완전히 대체하거나 병행하는 방향을 모색합니다. 초기 단계에서 Claude나 GPT-4를 쓰면 구현이 빠르고 비교적 정확할 것이며, 내부 시연 결과 Claude Vision API로 전환 시 정확한 3D 모델 생성 및 FPS 60 달성 등이 기대된다고도 분석되었습니다 [23] [22].

6. 도면 이미지 → JSON 데이터 추출: 프롬프트 설계와 스키마 구성

도면 인식에 LLM을 활용하는 경우, **시스템 프롬프트**와 **출력 데이터 스키마**를 정교하게 설계해야 일관된 JSON 결과를 얻을 수 있습니다. 이미 내부 구현에서 Claude API를 사용할 때 원하는 JSON 포맷을 예시로 제공한 바 있는데, 이를 발전시키겠습니다.

- **JSON 데이터 스키마**: 도면에서 추출해야 할 정보를 구조화하기 위해, 다음과 같은 스키마를 정의합니다 (내부에서 사용된 포맷을 기반으로 보완함):

```
{
 "rooms": [
  {
   "id": "room_1",
   "name": "거실",
   "vertices": [[x1, y1], [x2, y2], ...],
   "area": 25.5,
   "height": 2.8
  },
  ...
 ],
 "walls": [
  {
   "id": "wall_1",
   "from": [x1, y1],
   "to": [x2, y2],
   "thickness": 0.2,
   "height": 2.8
  },
  ...
 ],
 "openings": [
  {
```

```
    "id": "opening_1",
    "type": "door",
    "position": [x, y],
    "width": 1.0,
    "height": 2.1,
    "wall": "wall_4"
  },
  ...
 ],
 "entrance": "opening_1",
 "scale": {
  "pixels_per_meter": 100
 }
}
```

위 구조는 **내부에서 이미 정의된 JSON 형식**과 거의 유사하며 [33] [34], 다음 사항을 포함합니다: - **rooms**: 각 방의 코너 좌표(`vertices` 폴리곤), 면적(㎡), 천장 높이(m). 이름은 도면에 표기된 "거실/침실1" 등의 텍스트를 추출하거나, 없을 경우 "Room 1" 같이 자동 부여합니다. - **walls**: 각 벽의 시작/끝 좌표 (`from`, `to`), 두께와 높이(m). 벽 세그먼트를 개별 객체로 표현해서 이후 3D 생성에 활용합니다 [35] [36]. - **openings**: 문과 창문 등의 개구부로, `type` 필드로 구분하고, 도면상 위치 좌표와 폭/높이(m)를 가집니다. 추가로 어느 벽에 붙어있는지 `wall` 필드로 해당 벽 ID를 연결해두면 3D에서 정확히 뚫을 위치를 찾기 쉽습니다. (내부 예시에선 `room`으로 방과 연결했지만, 창/문은 벽과 연결하는 편이 논리적이므로 변경했습니다 [34].) - **entrance**: 여러 개의 문 중 현관 출입구를 가리키는 ID입니다. 도면에서 "현관" 혹은 "Entrance" 등으로 라벨링된 문이나 건물 외곽과 접한 문 중 하나를 선택합니다. 이 필드는 1인칭 시점 시작 위치 결정에 쓰입니다. - **scale**: 도면 이미지 상의 픽셀을 실제 미터 단위로 변환하는 비율입니다. 예를 들어 `pixels_per_meter: 100`이라면 도면에서 100px 거리가 실제 1m에 해당함을 의미합니다 [37]. 만약 도면에 축척이 표기되어 있다면 이를 인식해 설정하고, 없으면 임의로 1m=100px 등의 기본값을 가정하도록 합니다.

- **프롬프트 설계**: 위 스키마에 맞춰 LLM에게 응답을 요구하려면, **시스템 메시지**나 **샘플 프롬프트**를 잘 구성해야 합니다. 예시로 Claude나 GPT-4에 보낼 지시를 설계해보면:

  시스템: 당신은 건축 도면을 분석하여 구조 정보를 JSON으로 출력하는 전문가 AI입니다. 반드시 JSON 형식으로만 답변합니다.
  사용자: 다음은 건축 평면도 이미지입니다. 이 이미지를 분석하고 다음 JSON 형식으로 반환하세요:

  ```
  { "rooms": [...], "walls": [...], "openings": [...],
  "entrance": ..., "scale": ... }
  ```

  각 필드의 의미: - rooms: 방마다 id, 이름(한글 그대로 추출), 꼭짓점 좌표 배열(시계방향 또는 반시계방향), 면적(㎡), 높이(단위 m). - walls: 벽마다 id, 시작/끝 좌표, 두께(m), 높이(m). - openings: 문이나 창마다 id, type("door" 또는 "window"), 중앙 좌표, 폭(m), 높이(m), 연결된 wall id. - entrance: 건물 입구에 해당하는 door의 id. - scale: 이미지 픽셀과 실제 미터 환산 비율 (예: 1m당 100px). 참고: 도면에 표시된 **치수선/수치**를 읽어 실제 길이를 계산하세요. 벽 두께나 방 크기가 표시되어 있다면 반영하세요. 창문/문은 벽의 중간에 끊긴 부분으로 표시됩니다. 모든 좌표는 이미지상의 x, y 픽셀 좌표로 출력하세요. 정확한 수치가 도면에 없으면 일반적인 벽 두께 20cm, 문 폭 0.9m, 창문 폭 1.5m 등을 가정하고 높이는 기본 2.8m로 하세요. JSON 이외의 불필요한 설명은 하지 마세요.

위와 같이 **JSON 구조를 명시**하고 각 필드 요구사항을 설명해 줍니다 [33] [34]. 특히 "반드시 JSON 형식으로만 답변"이라는 지시와 **예시 JSON 템플릿**을 넣어 모델이 불필요한 말을 섞지 않도록 유도합니다. 내부 구현에서도 프롬프트에 JSON 예시와 `"정확한 측정값이 있으면 추출하고, 없으면 표준 규격 가정"`류의 지침을 넣은 바 있습니다 [38].

이런 맥락을 유지하며, 몇 가지 추가 개선을 합니다: - **프롬프트 토큰 최적화**: 도면 이미지를 base64로 바로 모델에 넣는다면 컨텍스트를 많이 차지하므로, API에서 이미지로 직접 보낼 수 있는 경우 그걸 사용하고 텍스트 설명은 최소화합니다. - **예시 제공**: 가능하다면 간단한 예시 도면과 그에 대한 정답 JSON 한두 개를 few-shot으로 넣으면 모델이 형식을 더 잘 따를 것입니다. 다만 컨텍스트 용량이 허용하는 선에서 합니다. - **오류 방지**: 모델 응답에 JSON 외에 설명이 붙으면, 파싱에 실패하므로 이를 방지하기 위해 "기타 설명 금지"를 강조합니다. 그래도 혹시 모를 경우를 대비해, 코드 상에서 `response.content.text.match(/\{[\s\S]*\}/)` 으로 중괄호 부분만 추출하여 JSON.parse하는 로직을 이미 사용 중입니다 [39]. 이를 그대로 유지하거나 보완합니다. - **현관 문 인식**: "entrance" 필드를 정확히 채우도록, 도면에 **현관/출입구** 글자가 있으면 그 방을 입구로, 또는 외부와 연결된 문 중 하나를 선택하도록 지시합니다. 모델이 판단토록 해야 하므로 예시에서 entrance 지정까지 보여주는 것이 좋습니다.

이러한 프롬프트 전략을 통해 Claude나 GPT-4에게 이미지를 분석하게 하면, **거의 원하는 JSON 스키마 그대로** 결과를 얻을 수 있을 것입니다. 응답 예를 들면:

```
{
 "rooms": [ { "id": "room_1", "name": "거실", "vertices": [[0,0],[500,0],[500,400],[0,400]], "area": 20.0,
"height": 2.8 }, ... ],
 "walls": [ { "id": "wall_1", "from": [0,0], "to": [500,0], "thickness": 0.2, "height": 2.8 }, ... ],
 "openings": [ { "id": "opening_1", "type": "door", "position": [250,0], "width": 0.9, "height": 2.1,
"wall": "wall_1" }, ... ],
 "entrance": "opening_1",
 "scale": { "pixels_per_meter": 50 }
}
```

이렇게 도면 한 장당 JSON 데이터를 얻게 되면, 이후 프로세스에서 3D 모델을 생성하고 편집 기능과 연계할 수 있는 **표준 데이터 형태**가 확보됩니다.

추가 고려: 혹시 LLM 결과에 오류가 있거나 자신감이 부족한 경우, 보완책으로 **사용자 피드백 루프**를 둘 수도 있습니다. 예컨대 모델이 "벽의 길이를 정확히 모르겠다"고 판단되면, JSON에 `uncertain: true` 같은 플래그나 경고 메시지를 달 수도 있는데, 이는 파싱 시 걸러내고 UI 상에 "도면 해석을 일부 확인해주세요" 식으로 표시하는 것입니다. 하지만 이상적으로는 자동으로 정확히 인식되는 것이 목표입니다.

결론적으로, **정교한 프롬프트 + 일관된 JSON 스키마**를 사용하면 LLM 기반 도면 인식의 정확도를 크게 높일 수 있습니다. 이후 이 JSON은 3D 생성 알고리즘의 입력으로 사용되어, **도면→3D 자동 변환**의 근간이 될 것입니다 [40] [41].

7. 3D 지오메트리 절차적 생성 알고리즘 (벽체, 개구부, 바닥/천장)

도면 분석으로 얻은 JSON 데이터를 토대로 **Three.js** 상에 3차원 구조물을 자동 생성하는 알고리즘을 설계합니다. 이 알고리즘은 **벽체 생성**, **개구부(창문/문) 삽입**, **바닥/천장 생성**의 단계로 구분됩니다.

- **벽체 Mesh 생성**: JSON의 `walls` 목록을 순회하며 각 벽 세그먼트를 3D 메쉬로 만듭니다. 주어진 `from` [x1,y1]과 `to` [x2,y2] 좌표를 통해 **벽 길이**를 계산하고 [35], **벽 두께**와 **높이**를 사용해 Three.js의 BoxGeometry(또는 PlaneGeometry를 extrude)로 직사각형 단면 벽을 만듭니다. 예를 들어:

```
const length = Math.sqrt((x2-x1)**2 + (y2-y1)**2);
const geometry = new THREE.BoxGeometry(wall.thickness, wall.height, length);
const material = new THREE.MeshStandardMaterial({ color: 0xcccccc });
const wallMesh = new THREE.Mesh(geometry, material);
```

이렇게 생성한 벽 mesh를 **중점**에 배치하고 벽 방향으로 **회전**시킵니다 [42]. 중점 좌표는 `(x1+x2)/2, (y1+y2)/2`이며, mesh의 `position.y`는 `wall.height/2`로 하여 바닥(0높이)에서 정확히 해당 높이로 올라오도록 합니다 [43]. 회전은 `Math.atan2(y2-y1, x2-x1)` 값을 `rotation.y`에 적용하여 벽이 두 점을 잇도록 방향을 맞춥니다 [44]. 이렇게 하면 2D 도면 평면의 선분이 3D 공간에서 두께와 높이를 가진 벽체로 형상화됩니다. 각 벽 mesh에는 `castShadow = true`, `receiveShadow = true`를 설정해 조명시 그림자를 드리우고 받도록 합니다 [36]. 재질(Material)은 기본적으로 연한 회색(콘크리트 느낌)으로 하되, 후에 벽지 텍스처를 입히거나 Vertex Colors로 칠하는 등 변경 가능성을 열어둡니다. 모든 벽 mesh는 `THREE.Group`(예: sceneGraph의 `wallGroup`)에 추가해 한꺼번에 관리할 수 있게 합니다 [45].

- **바닥 및 천장 생성**: `rooms` 목록을 이용해 각 방의 바닥면을 생성합니다. 간단하게는, 모든 방의 경계를 포괄하는 전체 영역으로 하나의 평면(PlaneGeometry) 바닥을 만들 수 있습니다 [2]. 실제 코드에서 bounds를 계산해 `PlaneGeometry(bounds.width, bounds.depth)`로 바닥을 생성한 부분이 있습니다 [2]. 그러나 이렇게 하면 벽 바깥 부분까지 바닥이 생길 수 있으므로, 가능하면 **방마다 개별 바닥 mesh**를 생성하는 것이 좋습니다. 구현 방법은, 각 방의 `vertices` 좌표 배열을 가지고 **Shape**을 만든 다음 `ShapeGeometry` 또는 `ExtrudeGeometry`를 사용해 두께가 아주 얇은 바닥판을 생성하는 것입니다. 예를 들어:

```
const shape = new THREE.Shape();
vertices.forEach(([x,y], i) => {
  if(i===0) shape.moveTo(x,0-y);
  else shape.lineTo(x,0-y);
});
const floorGeom = new THREE.ShapeGeometry(shape);
```

이런 식으로 하면 방의 다각형 모양 그대로 바닥 mesh를 얻을 수 있습니다. Y좌표 부호를 반대로 한 것은 Three.js의 2D 좌표계 보정(도면 y증가 방향과 3D y축 반대)입니다. 바닥 재질은 **MeshStandardMaterial**로 약간 밝은 회색 또는 선택한 마감재 텍스처를 적용하고, `receiveShadow = true`로 설정해 그림자를 받도록 합니다 [46]. 생성된 바닥 mesh의 `rotation.x = -Math.PI/2` (XY평면으로 회전) [47], `position.y = 0`으로 바닥면에 붙입니다. 만약 ExtrudeGeometry로 약간 두께를 줬다면 position.y는 0으로 두고 두께만큼 아래로 내려가게 extrude 설정을 할 수도 있습니다.

천장(ceiling)은 바닥과 거의 동일하게 생성하되, 높이만 방 높이로 올리면 됩니다. 각 방의 천장은 바닥 shape와 똑같은 평면으로 만들고 `position.y = room.height`로 설정합니다. 또는 간단히 방 높이가 모두 동일하다면 바닥과 동일한 shape geometry를 복사해 y=공통높이 위치에 배치하면 됩니다 [48]. 천장은 기본적으로 `MeshStandardMaterial`에 흰색이나 밝은 색으로 하고, `emissive`를 약간 줘서 빛을 반사하는 느낌을 내면 좋습니다 [48]. 천장은 기본적으로 `visible = true`로 두지만, 탑뷰 모드일 때는 `visible = false` 처리하여 보이지 않게 합니다 (이 부분은 뒤의 뷰 로직에서 상세 설명). 바닥/천장 mesh들도 `floorGroup`, `ceilingGroup` 등으로 그룹화하여 관리하면 편리합니다.

- **개구부 (문/창) 삽입**: 벽체 생성 후, JSON의 `openings` 목록을 처리하여 벽에 **구멍**을 뚫는 작업을 합니다. 개구부 정보는 중앙 `position` 좌표와 `width`, `height`, 그리고 해당 `wall`의 id를 가지고 있습니다. 우선 각 opening을 어느 벽 mesh에 적용할지 찾아야 하는데, `wall` id가 주어졌으므로 이를 통해 해당 벽 mesh 객체를 식별합니다 (벽 mesh 생성 시 삼각Map 등에 wallId->mesh를 저장해 둡니다). 그런 다음, 벽 좌표계에서 개구부의 상대 위치를 계산합니다. 벽 mesh를 생성할 때 기준이 된 from->to 중점 좌표와 angle을 알고 있으므로, opening의 `position` (글로벌 좌표계의 x,y)와 벽의 기준을 비교해 **벽 중심에 대한 오프셋**을 구합니다. 예를 들어 벽 중심점 `(midX, midY)`와 벽 회전 `angle`이 있다면:
- 개구부의 벽 방향 위치 = proj = `( (opening.x - midX) * cos(angle) + (opening.y - midY) * sin(angle) )`.
- 개구부의 수직 벽면 위치 = (보통 문 높이의 절반 등) z_offset = opening.height/2.

- 이렇게 2D 상 투영 거리 proj를 구하면, 벽 길이 방향에서 얼마나 떨어졌는지 알 수 있습니다. opening 폭의 절반(`width/2`) 만큼 좌우로 여유를 두고, 벽체 geometry에서 해당 구간만큼의 면적을 뚫으면 됩니다.

CSG(Constructive Solid Geometry) 활용: Three.js에서는 mesh에 구멍을 내는 방법으로 **CSG 기법**을 사용할 수 있습니다. 추천하는 라이브러리는 `three-bvh-csg`로, 빠르고 정확하게 CSG 연산을 수행합니다 [49]. 벽 mesh와 동일한 geometry 형태의 **절삭용 메쉬**(subtraction mesh)를 만들어 CSG로 빼주면 구멍이 납니다. 구체적으로, 문의 경우 **door shape**에 해당하는 박스형 mesh를 생성합니다. 폭은 opening.width, 높이는 opening.height, 두께는 벽 두께보다 약간 여유있는 값(예: wall.thickness * 1.1)으로 BoxGeometry를 만들고, 위치는 벽의 로컬좌표에서 앞서 계산한 proj 지점에 centered 되도록 놓습니다. 문은 바닥에서 0m 높이에 시작하므로 y=opening.height/2가 되고, 창문은 일반적으로 바닥에서 올라간 위치에 있으므로 만약 창문의 밑높이가 일정하다면 (예: 창문 하단 1m 상공) y=(창문밑높이 + 창문세로길이/2)로 설정합니다. 이러한 절삭 mesh를 준비한 후, `CSG.subtract(wallMesh, cuttingMesh)`를 수행하면 벽 mesh geometry에서 그 부분이 깎여 나갑니다.

`three-bvh-csg` 사용 예:

```
import { CSG } from 'three-bvh-csg';
const wallCSG = CSG.fromMesh(wallMesh);
const holeGeom = new THREE.BoxGeometry(open.width, open.height, wall.thickness*1.2);
const holeMesh = new THREE.Mesh(holeGeom);
holeMesh.position.set(holeLocalX, open.height/2, holeLocalZ);
const holeCSG = CSG.fromMesh(holeMesh);
const newWallCSG = wallCSG.subtract(holeCSG);
const newWallMesh = CSG.toMesh(newWallCSG, wallMesh.matrix, wallMesh.material);
```

이렇게 하면 newWallMesh가 구멍이 뚫린 새로운 mesh가 됩니다. 생성 후에는 원래 wallMesh를 scene에서 제거하고 newWallMesh로 대체합니다. 이 작업을 필요한 모든 opening에 대해 수행합니다. **주의**: CSG 연산은 연속으로 하면 geometry가 깨질 수 있으므로, 한 벽에 구멍 여러 개 뚫을 때는 한 번의 CSG.subtract에 모든 holeCSG를 차례로 빼거나, 루프로 돌리되 중간에 mesh 변환을 해줘야 합니다.

문과 창의 3D 표현도 고려합니다. 단순히 구멍만 내면 빈 공간인데, 여기에는 후속으로 실제 문짝이나 창틀 모델을 배치할 수 있습니다. 우선 자동화 단계에선 구멍만 내고, 나중에 사용자가 문/창 오브젝트를 선택해 끼워넣게 하거나 기본 문/창 프리팹을 삽입하는 식으로 할 수 있습니다. 하지만 최소한 **시각적 표시**로, 문 위치에는 문틀 형상의 얇은 박스(두께 벽과 동일, 프레임 느낌)나 회전 가능한 문짝(mesh)를 배치해두면 유저가 인지하기 쉽습니다. 이 부분은 가구 배치와 유사하게 볼 수도 있지만, 기본 구조로 넣어두는 게 편합니다.

요약하면, **벽체->바닥/천장->개구부 순으로 3D 메쉬를 구성**하며, 각 단계에서 적절한 Three.js API와 수학적 계산을 사용합니다. 이 알고리즘은 도면이 바뀌거나 사용자가 새로운 방을 추가해도 재실행하여 즉각 3D 갱신이 가능하도록 설계합니다. 전체 결과로, 도면 한 장만 넣으면 곧바로 해당 평면의 **3D 공간 뼈대(벽, 바닥, 천장, 문/창 구멍)**가 완성되며, 이는 이후 텍스처링이나 가구 배치의 무대가 됩니다 [50].

- **성능 및 정밀도 고려**: 벽체 개수가 많아도 BoxGeometry는 가볍고, CSG 연도 modern 버전을 쓰면 비교적 빠릅니다. 그러나 너무 세부 방위치까지 1cm 단위로 표현하려 하면 Vertex 수 증가로 비효율일 수 있으니, 좌표를 0.001 정밀도까지는 아니더라도 mm 단위는 버리는 등 스냅처리해도 됩니다. 또, 모든 벽을 한꺼번에 합쳐 하나의 Geometry로 만들 수도 있지만 (BufferGeometryUtils.mergeBufferGeometries 이용), 차후 **개별 벽 제거/투명 처리 등의 인터랙션**을 생각하면 mesh 단위로 놔두는 것이 유연합니다.

- **데이터 구조 연결**: JSON의 각 객체와 생성된 Three.js Mesh를 연결짓는 맵을 유지합니다 (`id->mesh`). 이것은 나중에 사용자 편집(벽 삭제 등)에 대응하거나, 하이라이트 표시, 속성 조회 등에 필요합니다.

8. 뷰 모드 전환 로직: 탑뷰(Top-Down) ↔ 1인칭(FPS)

3D 모델이 생성되면, 사용자에게 **두 가지 시점(View Mode)**으로 탐색/편집 기능을 제공합니다: **탑뷰 모드**(전체 평면을 위에서 내려다보는 CAD 스타일 뷰)와 **1인칭 모드**(공간 내부를 걷는 시점). 이 둘의 전환과 표시 요소 토글 로직을 설계합니다.

- **카메라 설정**: 기본적으로 R3F `<Canvas>` 내에 두 종류의 카메라를 설정하고 필요에 따라 전환합니다.
- **탑뷰 카메라**: 정사영 **OrthographicCamera**를 사용하여 왜곡 없는 2D 평면도를 볼 수 있게 합니다. OrthographicCamera의 left/right/top/bottom을 공간 크기에 맞게 설정하고, 카메라 위치는 평면의 중심 위쪽 (예: `position.set(centerX, maxDimension*1.5, centerY)`), rotation은 바로 아래를 향하도록 `rotation.x = -Math.PI/2` (또는 OrbitControls를 쓰면 target을 center로 두고 각도를 90도로 고정) 합니다. 정사영이 아니더라도, PerspectiveCamera를 멀리 올리고 fov를 좁게 주면 유사한 효과가 나지만, 정확한 CAD 느낌을 주려면 Orthographic이 낫습니다. 이 카메라로 진입 시 **전체 평면이 화면에 꽉 차도록** 줌을 맞추고, 사용자가 스크롤로 확대/축소하거나 드래그로 평면을 이동(pan)할 수 있게 OrbitControls의 제한을 겁니다. OrbitControls를 탑뷰에 쓰는 경우 `maxPolarAngle = minPolarAngle = Math.PI/2`로 고정해서 수직 각도만 유지하고, `enableRotate = false` (Y축 주위 회전 비활성) 등의 설정을 합니다. 이렇게 하면 마치 2D 플래너처럼 상하좌우 이동과 줌만 되는 뷰가 됩니다.

- **1인칭 카메라**: PerspectiveCamera를 사용하며, **사용자 눈 높이**에 해당하는 위치(예: y=1.6m)에서 시작하도록 합니다. 이 카메라는 이동에 따라 위치/rotation이 실시간으로 바뀔 것이므로 Three.js의 기본 컨트롤러 대신 **사용자 정의 컨트롤**(키보드/마우스 이벤트 + 물리엔진)로 조작합니다. 1인칭 진입 시 카메라의 초기 위치는 JSON에서 추출한 `entrance` 문 위치를 기준으로 실내 약간 안쪽을 향하도록 잡습니다. 예컨대 entrance opening의 좌표가 [x,y]라면, 그 문이 속한 벽의 내부 방향으로 1m 들어와서, 캐릭터 높이 1.6m로 설정합니다. 또한 카메라의 orientation(look direction)은 문 방향과 동일하게 또는 실내 중앙을 향하도록 약간 보정합니다. 이 초기 세팅은 사용자가 모드 전환 시 자연스럽게 현관으로 "들어오는" 느낌을 주기 위함입니다.

- **씬 요소 토글**: 두 모드에서 보여줄 요소와 감출 요소를 구분합니다.

- **탑뷰 모드**에서는 **천장(ceiling)**을 제거해야 내부가 보입니다. 따라서 천장 그룹 전체를 `visible = false`로 설정합니다 [48]. 또한 1인칭 캐릭터나 1인칭 전용 효과(예: 손이나 공구 모델 등)가 있다면 탑뷰에서는 숨깁니다. 반대로 **가구**나 **기타 편집 UI 요소(그리드 등)**는 탑뷰에서 적극 보여줍니다. 예를 들어 바닥에 **GridHelper**를 깔아 현재 스냅 격자를 표시하거나, 방 이름 태그를 떠있게 할 수도 있습니다. 벽이 너무 높아 탑뷰에서 방 내부 가구를 가릴 경우, 탑뷰 모드일 때는 벽의 높이를 일시적으로 낮추거나 반투명 처리하는 옵션도 고려할 수 있습니다 (예: 벽 height를 1m 정도로 줄여서 평면도처럼 보이게). 그러나 작은 높이는 여전히 가구를 가릴 수 있으므로, 차라리 **벽 투명도 조절 UI**를 제공해 사용자가 선택적으로 벽을 숨길 수 있게 하는 게 좋습니다. 우선 기본 구현에서는 천장만 제거해도 대부분 내용이 보일 것이므로, 벽은 그대로 두되 필요시 기능을 추가합니다.

- **1인칭 모드**에서는 천장을 보이게 하여 공간이 폐쇄되도록 하고, GridHelper나 편집 UI 등은 숨깁니다. 대신 **손전등 효과**나 **동적 그림자** 등을 켜서 분위기를 살릴 수 있습니다. 또한 탑뷰에서 배치한 2D UI (드래그 인터페이스 등)도 1인칭에선 대부분 숨기고, 최소한의 **인벤토리 HUD**나 크로스헤어만 표시합니다. 1인칭에서는 **광원** 설정도 다를 수 있는데, 탑뷰에서는 전체 조명이 골고루 켜져있었다면 1인칭에선 카메라 주변에 약한 광원을 붙여 어두운 곳을 볼 수 있게 하거나, 아니면 낮시간 설정으로 전체 밝게 유지할지 결정합니다.

- **전환 애니메이션**: 탑뷰 <-> 1인칭 전환 시 **부드러운 이동**이 중요합니다. 예를 들어 탑뷰에서 1인칭으로 갈 때, 카메라를 상공에서 현관 위치까지 자연스럽게 이동/회전 시키면 몰입감이 높습니다. 구현상으로는, 모드 전환 요청 시 현재 카메라 (Orthographic)의 뷰를 3D 공간의 한 애니메이션 경로를 따라 1인칭 카메라 위치까지 움직이는 연출을 합니다. OrthographicCamera에서 PerspectiveCamera로의 전환은 타입이 달라서 곧바로 보간하기 어렵지만, 대신 하나의 PerspectiveCamera를 사용하면서 탑뷰 모드일 땐 높은 곳+얕은 fov로 사용하고, 1인칭일 땐 낮은 곳+넓은 fov로 사용하는 방법도 있습니다. 그러나 값 전환이 복잡하니, 차라리 분리된 두

Camera를 두고, 모드 스위치 시 **CrossFade** 식으로 전환하는 기법도 고려됩니다 (R3F `<PresentationControls>`나 react-spring을 사용해 카메라 매트릭스를 보간 가능). 다소 구현 복잡성이 있으므로, 초기에는 전환 시 즉시 perspective 시점을 entrance로 텔레포트하고, 필요하면 Framer Motion이나 GSAP 등으로 약간의 fade-in/out 효과 정도만 주어도 무방합니다.

- **컨트롤 전환**: 모드 전환 시 **입력 제어**도 변경됩니다. 탑뷰에선 마우스로 패닝/줌 (OrbitControls)이나 클릭으로 오브젝트 선택/배치 등이 주로 쓰이고, 1인칭에선 키보드+마우스로 이동/시선 제어 (PointerLockControls + 물리)와 크로스헤어 대상 인터랙션이 주가 됩니다. 전환 시에는 다음을 수행합니다:

- 탑뷰→1인칭: OrbitControls 비활성 (혹은 카메라 detach), PointerLockControls 활성화. 마우스 커서를 중앙에 고정(lock)하고 숨깁니다 (브라우저 Pointer Lock API). 키보드 이벤트 리스너(WASD)를 등록하고, 물리 엔진의 캐릭터 바디를 활성화합니다. 또한 인벤토리 HUD를 화면에 표시하고, 크로스헤어를 보여줍니다.
- 1인칭→탑뷰: PointerLockControls 해제 (마우스 커서 해제하여 UI 클릭 가능), 키보드 이벤트 해제 (또는 무시), 물리 캐릭터를 비활성화 또는 중지. OrbitControls 재활성화하여 마우스로 평면 탐색 가능. 인벤토리 HUD와 크로스헤어 숨기고, 대신 탑뷰용 UI (가구 패널 등) 표시. 천장 숨김 등 그래픽 토글 적용.

- **시점별 추가 기능**: 탑뷰 모드는 **편집 중심**이 될 것이고, 1인칭은 **탐색/플레이 중심**이 될 것입니다. 따라서 탑뷰에서는 드래그로 가구 배치, 벽 변경 등의 편집 툴 버튼들이 화면에 나타나고, 1인칭에서는 문 열기/닫기, 오브젝트 상호작용(예: 가구 선택) 등의 플레이 기능이 활성화됩니다. 이러한 기능적 차이도 전환 로직에서 상태를 바꿔줘야 합니다 (예: `mode = "top"` vs `"firstPerson"`에 따라 UI 컴포넌트 렌더 분기).

- **스폰 및 충돌 고려**: 1인칭 모드로 들어갈 때 스폰(Spawn) 위치를 신중히 정해야 합니다. 현관 근처라고 해도 혹시 문이 막혀있다면? 기본적으로는 현관 문의 내부 조금 안쪽에 배치하지만, 만약 그 자리가 가구로 막혀 있으면 충돌 처리에서 튕길 수 있습니다. 그러므로 **스폰 시 충돌체크**를 해서 빈 공간이 확보된 지점인지 확인하고, 살짝 옆으로 치우는 등 보정해야 합니다. 또는 더 간단히, 스폰 직후 **일시적으로 충돌을 끄고** 몇 프레임 후 켜는 방법도 있습니다 (안전하게 플레이어가 비집고 나오도록).

- **멀티층 지원**: 현재는 단층 실내만 다루지만, 혹시 여러 층이 쌓인 건물이라면 탑뷰에서 층간 전환 UI를 두고, 1인칭에서도 계단을 통해 위아래로 갈 수 있도록 해야 합니다. 이 경우 천장 토글을 층별로 관리해야 하고, 1인칭 시작 위치도 현재 층 현관으로 지정해야 합니다. 복잡해지므로 우선 단층 전제로 구현하고, 확장 여지를 남깁니다.

전체적으로, **모드 전환 로직**은 사용자에게 두 가지 관점에서 맵을 다루는 경험을 제공하고, 각 모드에 적합한 UI와 컨트롤 환경을 제공하는 것입니다. 이를 통해 사용자는 **탑뷰에서는 설계자처럼** 공간을 편집하고, **1인칭에서는 게임 플레이어처럼** 공간을 체험할 수 있게 됩니다. 이 두 모드는 동일한 장면 데이터를 공유하며, 단지 표현과 조작 방법만 다를 뿐이라는 점에서, **상태 동기화**가 핵심입니다. 다음 단계에서는 이러한 상호작용과 동기화에 대해 구체적으로 다룹니다.

9. 1인칭 '마인크래프트 스타일' 상호작용 구현

1인칭 모드에서는 마치 마인크래프트와 유사한 방식으로 사용자가 환경과 상호작용할 수 있도록 설계합니다. 주요 요소는 **레이캐스터 기반 오브젝트 탐지**, **WASD 이동 및 점프 (물리엔진 사용)**, **인벤토리 시스템 통한 오브젝트 배치/제거**입니다. 각각의 흐름을 구체적으로 정의하면 다음과 같습니다.

- **레이캐스트(Raycast)와 커서**: 1인칭 시점 중앙에는 **크로스헤어**(십자선 모양 커서)가 표시되고, 사용자가 바라보는 방향으로 레이(광선)를 발사하여 정면의 오브젝트를 탐색합니다. Three.js의 `Raycaster`를 사용해 매 프레임 혹은 마우스 이동시 갱신하여, 크로스헤어 방향으로 일정 거리(예: 5m)의 충돌을 검사합니다.
- 만약 레이가 **상호작용 가능한 오브젝트**(예: 가구, 벽, 문 등)에 충돌하면, 해당 오브젝트를 하이라이트합니다. 하이라이트 방식은 Mesh의 material 색을 변경하거나, 선택된 객체 주위에 Drei의 `<Outline>` 효과나 `<Halo>` 등을 넣어 줄 수 있습니다. 또한 **커서 상태**를 바꿉니다. 예를 들어 평소엔 흰색 십자선이지만, 조준

대상이 있을 땐 녹색으로 바뀌거나 살짝 커지는 연출을 줍니다. 이렇게 하면 사용자가 "이 물체와 상호작용 가능"하다는 것을 즉각 알 수 있습니다. 질문에 언급된 "커서 상태 토글"이 바로 이 개념으로, 상호작용 가능 시 커서 모양/색을 변경하고 아니면 기본 상태로 되돌리는 것을 의미합니다.
- 레이캐스트 대상에 따라 다른 상호작용을 트리거합니다. 예를 들어 문에 조준하고 마우스 좌클릭하면 **문 열기/닫기** 액션을 취하거나, 가구에 조준하고 어떤 키(E 등)을 누르면 **가구를 집기/배치** 등. 이 부분은 추후 액션 매핑을 정의해야 합니다. 기본적으로,
    - **파괴 모드**(Minecraft의 블록 부수기처럼)일 경우: 조준한 오브젝트가 삭제 가능(예: 사용자가 배치한 가구나 벽의 일부라면)하면, 클릭 시 제거. 제거 후 서버/상태 동기화.
    - **설치 모드**일 경우: 인벤토리에서 선택한 아이템이 있으면, 레이가 닿는 지면이나 벽 위치에 **유령 오브젝트(프리뷰)**를 표시합니다. 그리고 클릭하면 그 지점에 실제 오브젝트를 배치 확정합니다.
    - 일반 모드에선 기본 상호작용 (문 열기 등)만.

- 레이캐스트 거리는 필요에 맞게 조절하되 너무 멀리까지 하면 부자연스럽고 성능도 낭비이므로 5~10미터 정도로 제한합니다 (집 내부 시야에서는 충분).

- **이동 및 물리 구현 (Rapier/Cannon.js)**: 1인칭 캐릭터(사용자)는 **물리엔진의 캐릭터 본체**로 구현해서 충돌 및 중력을 적용합니다. `@react-three/rapier`를 사용할 것을 권장하는데, 이는 Rapier(빠른 WASM 물리엔진)을 R3F에 통합해주는 훅입니다. Cannon.js를 사용해도 되나 Rapier가 성능이 더 좋고 유지보수 활발합니다.

- **캐릭터 바디**: 캡슐 모양의 콜라이더를 사용합니다 (반지름 = 어깨 반폭 약 0.3m, 높이 = 눈높이 ~1.6m). 이 바디는 **고정된 회전 (fixed rotation)** 설정을 해서 쓰러지거나 뒤집히지 않도록 합니다. 물리엔진 세계에서는 중력(-9.81m/s^2)을 적용하여 바닥에 착지하도록 하고, 벽이나 가구에 부딪히면 통과하지 못하게 충돌체로 막습니다. Rapier에서는 `RigidBody`에 `gravityScale`을 기본 1로 두면 중력 작용하며, `lockRotations=true`로 두면 회전안함, `collider`는 `capsule` shape로 생성합니다.
- **WASD 이동**: 키보드 이벤트로 W/A/S/D 키 입력을 받으면, 카메라가 가리키는 방향을 기준으로 **앞/뒤/좌/우** 벡터를 계산해 캐릭터 바디에 **속도(velocity)**를 부여합니다. 구체적으로, 매 프레임 키 입력 상태를 확인하여,
    - W: 카메라 로컬 방향 (yaw 방향)의 단위 벡터 * speed -> velocity.xz,
    - S: 앞의 반대(-forward),
    - A: 좌측 (forward 벡터를 -90도 회전),
    - D: 우측.
    - 동시에 W+D 등 대각 입력도 합성해서 대각선 이동 가능.
    - 속도 크기(scalar)는 달리기/걷기 설정 (예: 3 m/s).
    - Rapier에서는 `rigidBody.setLinvel({x: vx, y: currentVelY, z: vz})` 식으로 수평 속도를 설정하고, 수직 속도는 기존 값 유지(점프 아닌 이상).
- **점프 구현**: 스페이스바 누르면 캐릭터가 점프하도록, `rigidBody.applyImpulse({x:0, y: jumpForce, z:0}, true)`를 사용합니다 (Rapier 기준). jumpForce는 적당한 값 (예: 5~6)으로 캐릭터 질량에 맞춰 튜닝. 다만 연속 점프를 막기 위해 **바닥에 닿아있을 때만** 가능하게 해야 합니다. 이는 캐릭터 발 아래에 레이캐스트를 하거나, 또는 물리엔진에서 캐릭터와 바닥 충돌 이벤트로 접지 상태를 추적하여 구현합니다. Rapier의 `onCollisionEnter/Exit` 훅을 써서 지면태그와 충돌 여부를 확인할 수 있습니다. Cannon.js 경우 캐릭터 바닥에 sphere 하나 붙여서 접촉 검사하는 꼼수 쓰기도 합니다.
- **카메라-캐릭터 동기화**: 1인칭에서는 카메라가 플레이어 눈 역할이므로, 카메라는 물리바디의 움직임에 따라 위치를 업데이트해야 합니다. 구현 방법은, 카메라를 rigidBody에 `useFrame`으로 따라붙이거나, 카메라 자체를 물리엔진의 객체로 만들지(비권장). 일반적으로는 **카메라 객체를 물리바디의 자식**으로 두고 포지션 offset을 (0, 눈높이, 0)으로 설정합니다. 이렇게 하면 rigidBody 이동 시 카메라도 따라가고, 마우스 회전(PointerLockControls)이 카메라의 rotation을 바꾸더라도 물리바디의 방향과 sync를 맞출 수 있습니다.
- **Pointer Lock & 마우스 회전**: 마우스 움직임으로 시점을 회전시키는 부분은 Three.js PointerLockControls 코드를 참고하거나 custom 구현합니다. yaw(좌우 회전)는 캐릭터(=캡슐 바디 또는 카메라 y축)를 회전시키고, pitch(상하 회전)는 카메라의 local x축을 회전시킵니다. 각도 제한으로 pitch는 -89~89도 사이로 제한해 과도한 꺾임을 막습니다. yaw 회전 시 물리바디 자체도 따라 회전하게 하려면, rigidBody.setRotation(yawQuaternion) 식으로 적용하는 방법이 있으나, 캐릭터 콜라이더 회전이 중요한지

는 미미합니다 (캡슐은 축 회전 영향 없음). 그러나 움직이는 방향 계산을 위해 yaw값을 추적해야 하므로, 내부적으로 `playerDirectionAngle` 값을 업데이트하고 WASD 계산에 사용하면 됩니다.

- **중력 적용**: 물리엔진이 처리하므로 별도 로직은 필요 없습니다. 단, 비행 모드(없겠지만)나 계단/경사로에서의 처리 등이 과제인데, 일단 평지 가정입니다. Rapier에는 슬라이딩 처리나 auto step-up 같은 내장 기능은 없으므로, 계단이 있다면 간단히 경사 램프로 처리하거나, 접지감지+상승 보정 등을 해줘야 하지만, 범위를 넘습니다.
- **충돌 환경 구성**: 벽, 바닥, 천장, 가구 등 모든 고정 객체에는 `Collider`를 부여해야 캐릭터가 부딪힙니다. 벽과 바닥/천장은 생성 시 `Mesh`로 만들었으므로, `convex hull` 또는 `cuboid` 등의 콜라이더를 붙입니다. R3F Rapier에서는 `<RigidBody type="fixed">`로 static 바디를 만들고 그 하위에 `<mesh>`를 넣으면 자동으로 삼각메시 collider를 쓸 수 있지만, 삼각형 메시는 부정확하고 느릴 수 있어서, 벽 같은 건 **상자 콜라이더**로 직접 지정하는 게 효율적입니다. 다행히 우리 벽 mesh는 딱 직사각형 모양이므로, 그 두께/높이/길이에 맞게 Box 콜라이더를 만들면 됩니다. 혹은 `RigidBody`에 `colliders={false}`로 하고 `Rapier.createCollider` API로 위치/사이즈 지정해도 되고, simpler하게 `<RigidBody>` 태그 props로 shape사이즈 넘길 수도 있습니다.

- **월드 경계**: 플레이어가 바깥으로 떨어지지 않게, 바닥 plane 가장자리나 벽 바깥으로 못 나가게 해야 합니다. 벽이 건물 외곽을 에워싸고 있다면 문제없으나, 현관문 밖에 바로 떨어지는 공간이 있다면 보이지 않는 콜라이더로 막거나, 어차피 출입문으로 나가면 그냥 떨어지는 게 맞다면 Y<0 아래로 떨어지면 리스폰시키는 처리 등 정하면 됩니다.

- **인벤토리 시스템 & 오브젝트 배치**: 1인칭에서 마인크래프트 스타일로 블록(혹은 가구)을 놓고 부수는 핵심은 **인벤토리 HUD + 배치모드**입니다.

- **HTML 인벤토리 UI**: 화면에 **인벤토리 바**(예: 하단 중앙)나 별도 패널을 표시해 사용 가능한 아이템 목록을 보여줍니다. 마인크래프트처럼 단축키 1~9에 슬롯을 매핑해, 숫자 키로 빠르게 전환하게 할 수도 있고, 마우스로 눌러 선택하게 할 수도 있습니다. 이 인벤토리 UI는 React DOM으로 구현하며, Framer Motion 등을 이용해 1인칭 진입 시 슬라이드업으로 나타나고, 마우스 락 상태에선 보통 클릭 못하지만 인벤토리 열 때는 일시적으로 마우스 락 해제하거나, 키보드로만 아이템 전환하도록 할 수 있습니다. 편의상 기본 모드에선 단축키 전환만 지원하고, 별도 인벤토리 열기 키(I 등)를 누르면 마우스가 풀리면서 큰 인벤토리 창이 떠서 아이템을 drag&drop할 수도 있게 설계 가능합니다.
- **아이템 종류**: 프로젝트 특성상 **가구 아이템**들이 주된 대상입니다. 또한 벽지나 바닥재 같은 것도 블록처럼 발라넣는 기능이 있을 수 있지만, 우선은 테이블, 의자 등 가구 및 장식품 배치로 정의합니다. 인벤토리에는 여러 카테고리(가구, 타일, 조명 등)별 아이템이 있고, 1인칭 모드에선 주로 **블록 단위 배치**(마인크래프트처럼 벽돌 쌓기)는 없을 것이므로, 가구/소품을 들고 놓는 행위가 중심입니다.
- **배치 프리뷰**: 아이템을 선택하면, **placement mode**가 활성화됩니다. 이때부터 화면에 선택한 아이템의 **유령 모델(transparent ghost)**이 나타납니다. Ghost 모델은 Three.js로 미리 로드한 GLTF나 기본 형상의 clone으로 만들고, 재질을 투명+살짝 emissive로 설정하여 주변과 구분되게 합니다. Ghost는 레이캐스트를 이용해 현재 놓을 수 있는 위치에 따라 움직입니다.
    - Raycaster를 바닥이나 벽 등에 투사하여 충돌 포인트를 얻고, 그 지점에 아이템의 ghost를 이동시킵니다. 만약 바닥과 충돌했다면 그 지점에 놓고, 벽과 충돌했는데 아이템이 바닥에만 놓여야 하는 종류라면 ghost를 바닥로 투영(예: y좌표만 0으로)하거나, 벽걸이용 아이템이면 벽에 붙여둡니다.
    - **Grid Snapping**: 배치 시 정교함을 위해 격자 스냅을 적용합니다. 충돌 지점 좌표를 예를 들어 0.5m 단위로 반올림하여 ghost의 최종 좌표로 씁니다. 이렇게 하면 아이템들이 격자에 정렬되어 배치되어 마구 삐뚤어지지 않고 보기 좋습니다. Snapping 크기는 옵션으로 조절 가능하게 UI에 토글(격자 켜기/끄기)와 그리드 표시 연계합니다.
    - **회전**: 사용자는 배치 모드에서 R 키나 마우스 휠 등으로 아이템의 회전을 조정할 수 있습니다. 90도 단위로 회전시키거나, 자유롭게 연속 회전되게 할 수도 있습니다. 편의상 15°나 45° 스텝으로 회전하도록 하여 벽에 직교 배치 등 쉽게 맞춥니다. ghost 모델은 그에 따라 즉각 회전하여 미리보기 됩니다.
    - Ghost는 배치 가능한 위치인지에 따라 시각적 피드백을 줄 수도 있습니다. 예를 들어 다른 가구와 겹쳐서 둘 수 없는 곳이면 ghost 색상을 빨간색으로 바꾸거나, 땅에 닿지 않으면 반투명도 다르게. 이 겹침 판정은 Scene 내 다른 객체들의 bounding box와 ghost를 충돌 검사하거나 간단히 거리로 판단합니

다. 초기에 복잡하게 하기보다, 아예 사용자가 겹치게 두지 않도록 신뢰하거나, 추후 공간 충돌 체크를 넣어줄 수 있겠습니다.

- **배치 확정**: 사용자가 원하는 위치에 ghost를 움직였으면, **마우스 좌클릭**(혹은 다른 액션 키)을 통해 그 자리에 아이템을 놓습니다. 이때 ghost와 동일한 위치/회전으로 실제 아이템 Mesh를 복사 또는 로드하여 Scene에 추가합니다. 또한 **물리엔진**에 해당 아이템의 Collider를 생성하여 이제부턴 충돌의 일부가 되도록 합니다. (가구는 흔들리거나 움직일 필요 없으면 `type: fixed`로 배치). 데이터 측면에서는 Zustand store나 React 상태에 이 신규 객체의 정보(타입, 위치, 회전 등)를 추가합니다. 그리고 **Supabase나 전역 상태**에도 즉시 반영하여 persistent하게 저장될 수 있도록 합니다 (후술).
- **제거(파괴)**: 제거 모드에서는 마인크래프트 블럭 부수듯, 크로스헤어 대상이 제거 가능한 객체라면 클릭 시 Scene에서 제거합니다. 가구를 없앨 때는 진짜 부수는 애니메이션까진 아니더라도, 살짝 scale 다운 후 제거하거나 particle 효과를 줄 수 있습니다. 제거 후 store/DB에서도 삭제합니다. 벽이나 구조물을 1인칭에서 부수는 것은 일반 사용 시나리오는 아닐 것 같지만, 편집 모드라면 가능하게 할 수도 있습니다. (현 프로젝트에선 벽 배치를 마음대로 변경하진 않을 듯, 주로 가구만).
- **상태 전환**: 인벤토리에서 아이템 선택 -> 배치 모드 -> 배치 확정 or 취소 -> 기본 모드 로 돌아오는 흐름을 구현합니다. 배치 취소는 우클릭이나 ESC로 구현하여 ghost를 제거하고 모드를 빠져나옵니다.

- **UI 연계**: 1인칭에서도 간단한 UI (예: 현재 선택 아이템 아이콘, 개수 등) 보여주고 싶으면 Drei의 `<Html>`이나 raw HTML overlay를 쓸 수 있습니다. 하지만 전체화면 immersive 경험을 위해 최소한으로 두는 게 좋습니다.

- **전체 상호작용 플로우**: 위 요소들을 시간 순서로 하나의 사례로 묶어보겠습니다:

- 사용자가 1인칭 모드로 들어감 (Pointer Lock 활성, 크로스헤어 표시).
- 공간을 WASD로 돌아다니다가, 어떤 가구를 바라봄 -> 레이캐스트 히트 -> 해당 가구 outline/하이라이트, 커서 색 변화.
- `E` 키(예를 들면) 눌러 상호작용: 이 가구의 정보 패널을 띄우거나 들기. (Minecraft로 치면 블럭 쳤을 때 특별 동작)
- 사용자 인벤토리 단축키 `3` 눌러 소파 아이템 선택 -> 화면 하단 아이템 선택 표시, ghost 소파 나타남.
- 사용자가 빈 공간 바닥을 향해 바라보고 이동 -> ghost 소파가 바닥 위치에 스냅되어 따라다님.
- 적절한 위치에서 좌클릭 -> ghost 제거, 실제 소파 mesh 생성되어 그 자리에 놓임. 소파 collider 생성. State/DB에 추가.
- 실수로 벽을 막는다면, 소파를 다시 바라보고 하이라이트됨, `Delete` 키나 혹은 인벤토리에서 선택 후 우클릭 -> 제거 확정 -> 소파 mesh 삭제, DB에서 제거.
- 문을 바라보고 클릭 -> 문 rotation 열리는 애니메이션 실행 (hinge constraint나 simple rotation).
- 필요 작업 후 탑뷰 모드 전환 -> Pointer Lock 해제, 마우스 커서 복귀, 1인칭 관련 UI 숨김, 탑뷰 UI 보임, OrbitControls 사용 가능.

이러한 **마인크래프트 스타일 상호작용**을 구현하면, 사용자에게 **편집의 즐거움**과 **공간 탐험의 재미**를 모두 줄 수 있습니다. 기술적으로 핵심은 **레이캐스팅**과 **물리엔진 제어**이며, React 상태 관리(Zustand 등)를 잘 활용해 현재 모드와 선택 아이템 등을 전역 상태로 관리하면, 여러 컴포넌트(인벤토리 UI, 3D Ghost, Raycast logic)가 유기적으로 동작할 수 있습니다. 또한 멀티유저 상황에서도 이러한 상호작용을 다른 클라이언트에 실시간으로 동기화하려면, 아이템 배치/제거 시 서버에 바로 반영하고, 이를 구독 중인 다른 클라이언트가 적용하는 구조를 가져야 합니다 (다음 섹션에서 설명).

끝으로, 마인크래프트와 다른 점은 완전히 자유로운 파괴/건설 게임이 아니라 **인테리어 디자인 툴**이므로, 필요에 따라 제한을 둘 수도 있습니다. 예를 들어 구조벽은 제거 불가, 가구 배치만 가능 등 규칙을 정하고 UI에 안내하면 됩니다. 또는 **편집 모드**와 **탐험 모드**를 구분해, 편집 모드일 때만 배치/삭제가 가능하고 탐험 모드에서는 오브젝트와 상호작용(앉기, 문열기 등)만 가능하게 해도 됩니다. 이 기획은 UX 방향에 따라 결정할 사항이지만, 기술 구현은 위 플로우를 토대로 유연히 대응할 수 있습니다.

10. 탑뷰 모드 Drag & Drop 가구 배치 시스템

탑뷰 모드는 1인칭보다 **정밀한 편집**에 유리하므로, CAD 소프트웨어에 가까운 **드래그 앤 드롭 가구 배치 시스템**을 구축합니다. 여기서는 마우스 중심의 그래픽 인터페이스로 가구 등을 배치/편집하는 방법을 정의합니다. 주요 기능은 **가구 라이브러리 패널**, **마우스 드래그 배치**, **격자 스냅(Grid Snapping)**, **회전 및 삭제** 그리고 **1인칭 뷰와 데이터 동기화**입니다.

- **가구 라이브러리 패널(UI)**: 화면 한쪽(예: 좌측)에 카테고리별 가구 목록 패널을 표시합니다. 내부 문서에서 사이드 패널 구조에 대한 언급이 있었는데 [51], 이를 참고하여 디자인합니다. 패널 상단에는 **카테고리 탭** 또는 드롭다운이 있어 "가구 / 타일 / 벽지 / 조명" 등 선택이 가능하고, 선택된 카테고리에 해당하는 아이템 썸네일 리스트가 패널에 나타납니다 [51]. 각 아이템은 이미지와 이름으로 표시되며, 패널 하단에는 검색창이나 필터(예: 색상, 크기) 옵션이 있을 수 있습니다 (후순위 기능). 이 패널 UI는 React로 구성하고 CSS Grid나 Flex로 아이템들을 정렬합니다. 반응형 디자인도 고려하여, 모바일에서는 패널이 슬라이드로 열리거나 아이콘만 보여주는 등 최적화합니다.

- **Drag & Drop 배치**: 사용자가 패널에서 원하는 아이템 썸네일을 **클릭 & 드래그**하여 3D 캔버스로 끌어놓으면, 해당 위치에 아이템이 배치되도록 합니다. 구현은 HTML5 Drag events와 R3F 이벤트를 조합합니다. 예를 들면, 각 패널 아이템 `<div>`에 `draggable=true`로 설정하고, onDragStart 이벤트에서 아이템 타입 정보를 저장해둡니다. Canvas 영역은 드롭존이 되어 onDragOver, onDrop 이벤트를 받습니다. onDrop 발생 시 드롭 좌표를 알아야 하는데, 브라우저 드롭 이벤트에는 화면 좌표로 주어지므로, 이를 Three.js 세계 좌표로 변환해야 합니다. 방법은 `canvas.getBoundingClientRect()`로 캔버스 위치를 얻고, dropEvent의 clientX, clientY를 상대적 캔버스 좌표로 바꾼 다음, `renderer.xraycaster` 또는 `camera.unproject` 등을 사용해 바닥 평면과 교차시켜 3D 위치를 구합니다. 좀 복잡하므로, 대안으로 **패널 아이템 클릭 시 곧바로 ghost를 생성하고 마우스 모드로 전환**하는 방법도 있습니다 (1인칭 배치와 유사). 그러나 CAD 스타일에서는 실제 드래그 동작이 자연스럽기 때문에, drag & drop 구현을 도전할 가치가 있습니다.

일단 drag over 동안 **Ghost 미리보기**를 탑뷰에도 표시하면 좋습니다. 예를 들어 사용자 아이템을 드래그 중이면, 마우스 위치에 반투명한 해당 가구 모델을 따라다니게 합니다. 이때 ghost는 3D 객체이므로, onDragOver에서 마우스 좌표 -> 3D 평면 좌표 변환을 매번 해서 ghost.position을 업데이트합니다. Snap 기능도 여기 적용할 수 있는데, drop 전에 ghost를 grid 위치에 맞춰 보여주면 사용자가 놓기 전에 정확히 어디 들어갈지 알 수 있습니다.

드래그를 놓으면 (onDrop), ghost를 실제 객체로 확정 생성합니다. 이때 1인칭 배치 때와 비슷하게 Scene에 Mesh 추가, DB저장, state 업데이트를 합니다.

- **Grid Snapping**: 탑뷰에서는 눈금자(그리드)를 표시하며 배치 위치를 **격자에 정렬**시켜 정확도를 높입니다. Three.js의 `GridHelper`를 바닥에 표시하면 시각적 그리드를 쉽게 얻을 수 있고 [52], 이와 일치하는 간격으로 스냅 기능을 구현합니다. Snap 간격은 0.5m 등으로 정하고, 이는 UI에서 토글(끄기/켜기)와 간격 변경 옵션을 제공할 수 있습니다.
- 구현: 아이템을 배치하거나 이동할 때 좌표 `(x, y)`를 Snap간격 단위로 반올림합니다. 예: grid=0.5, x=2.73이면 2.5 혹은 3.0으로. y(또는 z좌표, 여기 평면상의)도 마찬가지.
- Snap은 바닥 평면에 대해서만 적용하도록 합니다 (벽 배치는 탑뷰에서 잘 안할테지만, 혹 벽걸이 아이템이 있다면 벽면 grid도 생각해야 하나 일단 제외).
- 마우스로 아이템을 움직일 때도 Snap 적용: 예를 들어 이미 놓인 가구를 드래그로 옮길 때, 움직이는 ghost를 grid에 붙입니다.

- UI 표시: GridHelper는 기본, Snap on/off 토글 버튼 (아이콘으로 자석 모양 등) 제공. Snap 켜짐 시 Panel 아이콘에 강조를 주거나 툴팁으로 안내.

- **아이템 회전**: 배치된 아이템을 선택했을 때 **회전 핸들** 또는 **키보드**로 회전시킬 수 있도록 합니다. CAD에서 회전은 오브젝트 선택 후 모서리에 회전 핸들이 나타나 드래그하여 돌리는 형태가 많습니다. 우리 경우 간소화하여:

- 아이템 선택 시, 해당 객체 주위에 **회전 제어 UI**(예: 둥근 원) 표시하고, 이를 드래그하면 회전. 이건 구현 복잡하니, 대신 **키보드 R키**를 누를 때마다 90°씩 회전하거나, UI 패널에서 회전 버튼 (↻)을 누르면 45° 회전시키는 식으로 해도 됩니다.
- 각 아이템은 `rotation.y` (수평면에서) 값만 조절하면 되며, Snap을 적용해 15° 혹은 45° increments로 제한하는 것이 깔끔합니다.

- 회전 조작은 1인칭보다 탑뷰가 좌표파악이 쉬우므로, 탑뷰에서 미리 방향을 맞춰놓고 1인칭에서 확인하는 흐름이 자연스럽습니다.

- **아이템 이동 및 삭제 (Drag & Delete)**: 이미 놓인 가구를 **재배치 또는 제거**할 수 있어야 합니다.

- **이동**: 사용자가 가구를 클릭 선택하면 해당 객체를 하이라이트 (예: outline)하고, 마우스로 드래그하면 그 객체가 따라 이동하도록 합니다. 구현은, R3F의 `onPointerDown`으로 선택->상태 저장(currentSelectedId 등), `onPointerMove`에서 if dragging that id, update its position = raycast(mouse) snap, `onPointerUp`에서 fix position and update state/DB.
- 드래그하는 동안은 ghost를 움직이는 것과 유사하게, 실제 mesh를 반투명 처리하거나 별도 ghost를 써도 됩니다. 간단히 하려면 mesh자체 위치를 실시간 변경하면서 따라다니게 하고, 놓으면 finalize.
- **멀티선택**: 향후 여러 아이템을 그룹으로 옮기는 기능도 생각할 수 있지만, 우선 단일선택만.
- **삭제**: 아이템이 선택된 상태에서 Delete 키를 누르거나, 또는 UI 상 휴지통 아이콘 클릭 시 제거됩니다. 이때 confirm 없이 바로 삭제하거나, 한 번 물어볼 수 있지만 귀찮을 수 있으니 즉시 삭제도 괜찮습니다 (undo 기능 없으면 확인 한번 하는게 안전). 삭제 구현은 1인칭에서와 동일하게 Scene에서 mesh 제거, state/DB에서 제거.

- **UI**: 선택 시 사이드 패널이나 상단에 선택된 아이템 이름 및 옵션(회전, 삭제 버튼 등)을 띄워줄 수 있습니다 [53]. 예를 들어 "선택된 아이템: 소파 색상: ■ 크기: 슬라이더 [삭제]" 이런 패널을 떠서 속성 편집도 가능하게 확장할 여지가 있습니다 [53]. 우선은 삭제/회전 정도만.

- **1인칭 뷰와의 동기화**: 탑뷰에서 배치/편집한 모든 변화는 **실시간으로 1인칭 뷰에 반영**되어야 합니다. 이는 동일 애플리케이션 내에서는 당연히 같은 Scene Graph를 공유하므로, **동일 상태를 바라보고 있기만 하면** 자동입니다. 예를 들어 Zustand store에 가구 목록이 있고 탑뷰에서 항목 추가/삭제/변경 시 store를 업데이트하면, 1인칭 모드의 렌더링 컴포넌트도 그 변화를 감지해 새 객체를 나타낼 것입니다.

- React Three Fiber에서는 state->JSX 리렌더 패턴으로, 예컨대 `furnitureList.map(item => <FurnitureMesh key={item.id} {...item}/>)` 이런 형태일 것이므로, state 변경시 해당 컴포넌트가 추가/삭제/업데이트 됩니다. 이 때 1인칭/탑뷰는 카메라와 UI만 다르고 Scene은 공유한다고 가정하면 한 쪽에서의 변화가 바로 다른 뷰에도 나타납니다.

- 기술적으로, 굳이 두 Scene으로 분리하지 않고 하나의 Scene에 두 카메라를 놓고 모드에 따라 하나만 렌더링하게 할 수도 있습니다. 그러나 R3F 구조상 두 개의 `<Canvas>`를 쓰거나, 아니면 `<Canvas>` 하나에 camera 조건부 변경 등을 하는 식이 될텐데, 구현 난이도 따라 결정합니다. 사실 Scene 공유로 인한 제어 복잡성 때문에, **두 개의 Canvas (각각 다른 camera)**를 쓰고 state만 공유하는 것도 고려됩니다. 이 경우 1인칭 Canvas와 탑뷰 Canvas 둘 다 동일 Zustand store를 구독해 가구 리스트를 그리고, 전환 시 하나 Canvas를 hide/show 하면 됩니다. 두 Canvas가 존재해도 state로 동기화되면 결과는 동일하게 보이므로, 방법 선택은 구현 편의나 성능 보고 결정합니다.

- **Supabase 기반 저장/공유**: 탑뷰에서 이루어진 편집 내용은 **프로젝트 데이터**로서 백엔드에 저장되어야 합니다. Supabase를 백엔드로 사용하기 때문에, 다음과 같은 구조를 둡니다:

- **Projects 테이블**: 프로젝트 메타데이터 (id, user_id, name, created_at 등).
- **FurniturePositions 테이블** (또는 JSON 컬럼): 각 프로젝트에 속한 가구 객체들의 속성 (id, project_id, type, pos_x, pos_y, pos_z, rot_y, scale 등). 벽 등 구조는 도면에서 생성되므로 별도로 저장할 필요 없고 (도

면 원본만 저장하면 재생성 가능), 사용자 배치 요소만 저장하는 식입니다. 또는 모든 것을 **하나의 JSON**으로 통합해 Projects 테이블의 `state_json` 필드에 넣는 방법도 간단합니다. 이 JSON에는 도면 기반 구조 + 배치된 가구까지 모두 포함해서, 프로젝트 불러오면 한 번에 parse->생성합니다. 내부에 이미 types/database.ts와 schema.sql 등이 있는 걸 봐선 어느정도 테이블이 정의된 듯합니다 [54].

- **API 또는 직접 DB 호출**: Supabase JS SDK를 사용하여 클라이언트에서 DB에 직접 `insert/update/delete` 호출할 수 있습니다 [55]. 예를 들어 새 가구 배치시 `supabase.from("furniture").insert({...})`를 하고, 삭제 시 `delete().match({id})`를 실행합니다. Realtime 기능을 쓰려면 테이블에 realtime 설정을 해야 하고, Supabase Realtime은 기본적으로 row-level로 Websocket 메세지를 지원합니다. 우리는 collaborative 편집을 위해 이를 켤 예정입니다.
- **저장 타이밍**: 사용자가 편집할 때마다 바로바로 서버에 반영할 수도 있고, 또는 로컬에 두었다가 "저장" 버튼 누르면 한번에 보낼 수도 있습니다. **실시간 협업**을 생각하면, 즉각 반영이 맞습니다. 이때 과도한 트래픽 문제는 보통 크지 않지만, 연속 드래그로 가구 이동 중 매 픽셀마다 update하면 초당 수십건 날아갈 수 있으니, **쓰로틀링/디바운스** 전략을 적용합니다. 예를 들어 드래그 중엔 로컬 위치만 바꾸고, 드래그 끝난 시점에 최종 위치 한 번 업데이트하거나, 500ms 주기로 최종 위치 업데이트.
- **로드**: 사용자가 프로젝트를 열면 (예: /projects/:id 페이지), Supabase에서 해당 프로젝트의 도면 이미지 + 저장된 JSON/객체들을 불러와서, 먼저 도면을 3D 생성 알고리즘으로 벽/바닥을 만들고, 이어서 저장된 가구목록을 iterate하여 각 아이템 mesh를 배치합니다.

- **버전 관리**: 편집이 누적되다 보면 undo/redo나 버전 저장 욕구가 생길 수 있지만, 우선 MVP에서는 단일 상태 저장으로 충분합니다. 추후 프로젝트 스냅샷 테이블 등을 두면 좋습니다만.

- **커뮤니티 공유 및 멀티유저 접속**: Supabase를 활용해 **프로젝트 공유**와 **실시간 멀티플레이**도 지원합니다.

- **공유**: 사용자가 자신의 프로젝트를 "공개" 설정하면, 다른 유저들이 이를 열람하거나 심지어 편집 체험할 수 있게 합니다. 구현은 Projects 테이블에 `is_public` 필드 두고, 공개된 프로젝트 목록을 별도 페이지에 나열 (커뮤니티 갤러리). 다른 사용자가 해당 프로젝트를 열면, 편집 권한은 없고 1인칭 탐색만 가능하게 할지, 아니면 포크하거나 협업하도록 할지 결정해야 합니다. 아마도 **동시 협업** 시나리오를 염두에 두는 것 같습니다.
- **멀티유저 접속**: 여러 사용자가 하나의 프로젝트에 동시에 접속하여 협업 편집하거나, 최소한 같이 둘러볼 수 있게 합니다. Supabase Realtime을 이용하면, 특정 프로젝트 room에 채널을 만들어 **presence(존재) 정보**와 **state 변경**을 브로드캐스트할 수 있습니다.
    - 방법1: Supabase Realtime의 Postgres CDC(Change Data Capture)를 사용, 테이블에 변화가 생기면 (insert/update/delete) 지정 채널로 이벤트를 보냅니다. 클라이언트 A가 가구 하나 추가하면 supabase realtime 통해 B, C에게 그 row insert 이벤트가 가고, 각자 클라이언트에서 이를 subscribe하여 동일 추가를 반영합니다. 이러한 **데이터 동기화**로 협업 편집이 가능합니다. 이미 Supabase 관련 문서에 "Realtime presence and collaborative editing"을 다루고 있을 것입니다 [12].
    - 방법2: Supabase의 real-time channel API를 사용, JS에서 `supabase.channel("project-123").send({type: 'broadcast', event: 'move', payload: {objectId, newPos}})` 등으로 임의 메시지를 보내고 받을 수도 있습니다. 이는 DB를 경유하지 않아도 되므로 빠르지만, 서버 state와 분리되니 편집결과 저장하려면 결국 DB반영도 필요해서, CDC 기반이 더 일원화된 흐름입니다.
    - **Avatar 표시**: 여러 사용자가 들어와 있다면, 서로의 존재를 인식시키기 위해 각 사용자 위치/시점을 공유할 수 있습니다. 간단히, 각 사용자가 자신의 **캐릭터 위치와 방향** 정보를 주기적으로 (예: 10Hz) 특정 realtime channel의 presence 상태나 broadcast로 보내고, 다른 클라이언트는 받아서 그 위치에 대표 캐릭터(예: 간이 아바타)를 움직이게 할 수 있습니다. Supabase Realtime에 Presence 기능이 있으므로 (동접 유저 추적), 이를 사용하면 자동으로 참가/퇴장 알림과, 상태 payload 공유가 가능합니다. 예를 들어 `channel.track({x:..., y:..., name:"user1"})` 하면 presence list에 상태가 실시간 업데이트됩니다.
    - 편집 충돌 처리: 만약 두 명이 동시에 같은 가구를 움직이면 어떻게 할지 등 규칙이 필요합니다. optimistic locking이나, 마지막 동작 우선 등 정책을 정해야 하나, 우선 협업 편집이 복잡하다면 **한 번에 한 명만 편집 모드, 나머진 보기 전용**으로 제약을 둘 수도 있습니다. 또는 Google Docs 처럼 객체별

lock을 둘 수도. 초기엔 사용 사례가 많지 않을테니 심플하게 최근 변경으로 덮어쓰는 방식으로 두고, 심각한 충돌은 피하도록 안내 (ex: "동시에 편집시 마지막 저장이 적용됩니다").

- **음성/채팅**: 커뮤니티 공유라면 채팅도 고려되나, 범위를 넘어가므로 생략.

- **시스템 아키텍처 통합**: 이러한 기능들을 전체적으로 지원하려면, 프론트엔드와 백엔드 구조를 정리해야 합니다. **Supabase**는 이 모든 것(Auth, DB, Realtime, Storage)을 커버하는 BaaS로,

- Auth: 이메일/소셜 로그인 등 사용 (이미 authStore.ts 등이 구현된 듯 [56]).
- Storage: 도면 이미지 파일 업로드에 사용 [57] (floor-plans 버킷).
- DB: projects, furniture 등 테이블 사용.
- RLS: Row-Level Security를 통해 본인 프로젝트만 쓰도록 권한 제어 (supabase-setup.md에서 RLS, ACL 설명이 있었습니다 [12]).
- Edge Functions: 만약 AI 해석 (Claude API 호출 등)을 서버에서 하려면 Supabase Edge Function (Deno runtime)을 써서, 이미지->AI->JSON 변환 API를 만들 수 있습니다. 하지만 Next.js API route(`app/api/analyze/route.ts`)에서 직접 해도 되니 선택입니다 [58].
- Realtime: 위 편집 동기화에 사용.

프론트엔드는 Next.js 앱으로, 폴더 구조 예시는 후술하겠습니다 (architecture).

11. 시스템 아키텍처 및 기술 스택 종합 설계

위에서 논의한 모든 기능을 통합하여, **시각적 완성도**와 **도면 인식 정확도**를 최우선으로 달성하는 시스템을 Zero-to-One 관점에서 설계합니다. 핵심은 **프론트엔드**(React Three Fiber 앱)와 **백엔드**(Supabase)를 유기적으로 연결하고, 개발 생산성을 위해 적절한 **폴더 구조와 모듈화**를 적용하는 것입니다. 또한 프로젝트를 효율적으로 구현하기 위한 **단계별 계획**과 사용하는 **기술 스택**을 정리합니다.

폴더 구조 (Next.js + R3F 예시)

프로젝트는 모노레포 형태로 구성되었을 수도 있으나, 여기서는 Next.js 13+ (App Router) 기반 프론트엔드 하나와 Supabase BaaS 연동으로 생각합니다. 주요 폴더 구조 예시는 다음과 같습니다:

```
/app
  ├─ page.tsx                  # 랜딩 페이지 (소개/시작 페이지)
  ├─ projects/
  │   ├─ create/page.tsx       # 새 프로젝트 생성 페이지 (도면 업로드)
  │   ├─ [id]/editor/page.tsx  # 편집 페이지 (R3F Canvas, 탑뷰/1인칭 모드)
  │   ├─ [id]/viewer/page.tsx  # 뷰어 페이지 (공개 프로젝트 1인칭 관람용, 편집불가)
  ├─ api/
  │   └─ analyze/route.ts      # 도면 분석 API 라우트 (이미지 -> JSON via AI) [58]
/components
  ├─ canvas/
  │   ├─ EditorScene.tsx       # R3F 3D 씬 컴포넌트 (공간 생성 및 모든 3D 자식 포함)
  │   ├─ WallMesh.tsx          # 벽 Mesh 컴포넌트 (props: wall data)
  │   ├─ FloorMesh.tsx         # 바닥 Mesh 컴포넌트
  │   ├─ CeilingMesh.tsx       # 천장 Mesh 컴포넌트
  │   ├─ FurnitureMesh.tsx     # 가구 Mesh 컴포넌트 (props: item data) [8]
  │   ├─ PlayerController.tsx  # 1인칭 플레이어 제어 (카메라/물리 연결)
  │   ├─ TopViewCamera.tsx     # 탑뷰 카메라 세팅 & controls (Orthographic)
  │   ├─ FirstPersonCamera.tsx # 1인칭 카메라 세팅 (Perspective + pointer lock)
```

```
  │   ├─ GridOverlay.tsx        # GridHelper 및 좌표 축 등
  │   └── ... (기타 필요한 Three.js 관련 컴포넌트)
  ├─ ui/
  │   ├─ Sidebar.tsx            # 좌측 사이드바 (아이템 카테고리/목록) 51
  │   ├─ InventoryBar.tsx       # 1인칭 하단 인벤토리 바
  │   ├─ Toolbar.tsx            # 상단 툴바 (저장버튼, 모드전환 토글 등)
  │   ├─ Modal.tsx              # 모달 창 컴포넌트 (예: 도움말, 설정 등)
  │   ├─ AuthPanel.tsx          # 로그인/회원가입 패널 (필요시)
  │   └─ Navigation.tsx         # 상단 네비게이션 (프로젝트 목록, 프로필 등) 59
/lib
  ├─ floorPlanAnalysis.ts       # 도면 AI 분석 모듈 (이미지 -> JSON via API) 60
  ├─ threeCSG.ts                # CSG 관련 유틸 (구멍 뚫기 기능 래핑)
  ├─ supabaseClient.ts          # Supabase JS 클라이언트 생성 및 설정 57 59
  ├─ physics.ts                 # 물리엔진(Rapier) 초기화 및 helper (전역 world 관
리)
  └─ constants.ts               # 각종 상수 (e.g., default wall height, speeds)
/store
  ├─ projectStore.ts            # Zustand store - 현재 프로젝트의 전체 상태 (방/벽/가
구 데이터 등)
  ├─ uiStore.ts                 # UI 상태 (현재 모드, 선택 아이템, 로딩중 여부 등)
  ├─ authStore.ts               # 사용자 로그인 상태, 프로필 등 56
/utils
  ├─ calculateBounds.ts         # 방 리스트 -> 전체 Bounds 계산 (floor size) 2
  ├─ snap.ts                    # 스냅핑 연산 util (pos->snapped pos)
  ├─ fileToBase64.ts            # 파일 업로드 유틸 (도면 이미지 base64 변환 등)
  └── ...
/public
  ├─ assets/...                 # 정적 에셋 (아이콘, 환경맵, etc.)
/types
  ├─ database.ts                # Supabase DB 타입 정의 (자동 생성) 61
  ├─ floorPlan.d.ts             # FloorPlanAnalysisResult 등 타입 정의
  ├─ project.d.ts               # Project, WallData, FurnitureData 등 타입
  └── ...
```

위 구조는 권장 예시이며, 실제 프로젝트에서는 docs/, test/ 등 폴더도 있을 수 있습니다. 중요한 것은 **canvas 컴포넌트와 UI 컴포넌트의 구분**, **상태 관리(store)** 분리, **유틸/라이브러리** 분리입니다. 이미 내부에 문서화된 architecture.md, supabase-setup.md 등이 있어 이와 크게 다르지 않을 것입니다 11 12 . 실제 find 결과를 보면 WallManager.tsx, floorPlanAnalysis.ts, furnitureStore 등 유사한 구조가 보입니다 60 17 .

- **WallManager vs WallMesh**: 기존에는 WallManager가 벽 생성을 일괄 담당했지만, 모듈성을 위해 벽 단위 컴포넌트를 만드는 것을 제안합니다. EditorScene에서 JSON의 walls 배열을 map 돌며 `<WallMesh key={id} wall={wallData} />`를 렌더링하면, WallMesh 내부에서 geometry 생성 및 mesh 반환을 하도록 하는 식입니다. 상태변화 시 React diff로 추가/삭제도 쉽게 관리되죠. (WallManager로 메모리 최적화 등을 하려 했다면, 그 역할은 util로 빼거나, 어차피 벽 많아봐야 수십개일테니 이 방식도 문제 없습니다).
- **FurnitureStore**: furniture도 projectStore에 넣을 수 있으나, 분리된 store로 관리하면 성능상 이점이 있습니다 (필요한 컴포넌트만 구독). 예를 들어 useFurnitureStore로 가구리스트만 관리하고, useProjectStore로는 벽/방 등 구조 + 기타 프로젝트 설정 관리. Zustand는 slice 패턴으로도 쓸 수 있습니다.
- **라우팅**: Next13 App Router 기준, projects/[id]/editor 경로를 편집 화면으로 하고, projects/create는 새로 도면 올리는 flow, viewer는 읽기용. 적절히 설정합니다.

- **API Route**: /api/analyze/route.ts – Next.js API에서 Supabase Edge function 대용으로 써도 되고, Edge Function에 구현해도 됩니다. 내부 문서에는 Claude API키 발급, .env 설정, floorPlanAnalysis.ts 교체 등의 가이드가 있었습니다 [62], 우리도 .env.local에 AI API 키들 (OpenAI/Anthropic) 저장하고, 서버 측 호출에 사용합니다.

기술 스택 정리

프론트엔드: - **React 18+ / Next.js 13**: React 기반 SPA/SSR 프레임워크. App Router로 파일 기반 라우팅, API Routes 활용. - **TypeScript**: 모든 코드에 TS 적용하여 타입 안정성 확보. Supabase DB 타입은 `types/database.ts`로 자동 생성하여 사용 [61]. - **React Three Fiber (R3F)**: Three.js를 React스럽게 사용. 이를 통해 3D 씬을 선언적으로 구성하고 React 상태와 쉽게 연결. - **Three.js**: 저수준 3D 엔진. R3F 내부에서 사용되며, 커스텀 쉐이더나 geometry 조작 등에 직접 활용. - **@react-three/drei**: R3F 보조 컴포넌트 모음. 카메라/컨트롤, 로더, 환경 등 편의 기능 활용 [15]. - **@react-three/postprocessing**: Drei Effects로 대체가능하지만, 종합적 후처리 지원. Bloom, SSAO, DOF 등 적용. - **@react-three/rapier (or cannon)**: 물리엔진 바인딩. Rapier 추천 (성능 우수). React hook으로 물리 세계 구성하고 RigidBody, Collider 컴포넌트 사용. - **Zustand**: 전역 상태관리. React Context보다 가볍고, redux보다 단순. 프로젝트 데이터와 UI 모드 등을 관리. 내부 사용 흔적 존재 [63]. - **Tailwind CSS**: 스타일링 프레임워크. 클래스명으로 빠르게 UI 구성. 내부 코드에서 `text-stone-400`, `hover:shadow-md` 등의 utility class가 보여 이 프로젝트에 Tailwind를 쓰는 것으로 추정됩니다 [64]. Tailwind로 반응형 대응도 쉽고, dark mode 지원도 용이합니다 [65]. - **Framer Motion**: React 애니메이션 라이브러리. UI 패널 슬라이드, 페이드 효과, 카메라 전환 등 부드러운 애니메이션 부여. - **React DnD**: Drag & Drop 구현에 `react-dnd` 같은 라이브러리를 써도 됩니다. 하지만 HTML5 DragEvents로 간단히 처리 가능하면 굳이 안 써도 됩니다. react-dnd는 복잡한 시나리오에 유용. - **React Icons** (또는 Radix UI 등): UI 아이콘 및 기본 컴포넌트. 버튼, 모달 등 가져다 쓸 수 있음. 디자인 일관성을 위해 DaisyUI나 ChakraUI 고려 가능하나, 3D 앱 특성상 커스텀 스타일이 많아 Tailwind로 자유롭게 하는 편이 좋을 듯.

백엔드 (BaaS): - **Supabase (PostgreSQL + Auth + Storage + Realtime)**: 프로젝트 DB 및 사용자 관리. 이미 세팅된 것으로 보임 [12]. 우리는 Supabase를 single source of truth로 활용. - Auth: Supabase Auth (JWT). Next.js Middleware로 세션 유지. `createSupabaseClient()`로 클라이언트 생성하여 auth, DB 호출에 사용 [57]. - Database: Supabase Postgres. 테이블: projects, furniture, (rooms/walls는 안가질듯), users 등. RLS 적용으로 보안. - Storage: 도면 이미지 저장. URL 확보하여 AI 분석에 사용. 혹은 base64로 바로 보내도 된다면 storage 안 거쳐도 되나, 기록 위해 올림. - Realtime: Broadcast/Presence 사용해 동시 편집 반영. `supabase.channel().on('postgres_changes', ...)` 활용. 또는 useEffect로 `.subscribe()`하여 events 수신. - Edge Functions: (Optional) Supabase edge function to call AI API (but Next API route suffice). - **Anthropic Claude API / OpenAI API**: 도면 분석에 사용. 서버에서 HTTP 호출로 이미지+prompt 보내 JSON 응답 받음. 비용과 rate-limit 고려해, 토큰 optimize. - **Poly/Asset libraries**: 가구 3D 모델이 필요하면, Polyhaven/Sketchfab 등 CC0 소스에서 glTF 가져와 asset pipeline에 넣습니다. 프로젝트 시작시 몇 개 다운로드받아 /public/assets/models/ 에 넣어두고, useGLTF로 로드.

DevOps & ETC: - **Vercel**: Next.js 호스팅 (Supabase와 궁합 좋음). Supabase DB와 연결. - **Git**: 버전관리. PRD, architecture 문서 있었고 commit logs 보이는 부분 있어 Git flow 사용 중으로 추측. - **ESLint/Prettier**: 코드 품질 도구. - **Playwright**: E2E 테스트 (if time). 3D 앱 테스트는 제한적이지만, UI flows (프로젝트 생성->배치->저장) 정도 시나리오 자동화 가능. - **Lighthouse**: 성능 측정, 85점 이상 목표가 있었음 [7]. 성능 최적화: code splitting, Suspense, lazy loading models, texture compression, etc. - **Security**: Supabase RLS, XSS 방지 (React default), CSRF -> if using Supabase Auth with cookies, ensure CSRF token on form (auth-utils addCsrfHeader 언급있었음 [66]).

핵심 컴포넌트 및 모듈 설명

- **EditorScene.tsx**: R3F Canvas 내부의 메인 3D 씬. 여기에 `<Physics gravity={...}>`로 Rapier 물리 영역을 감싸고, `<Suspense>`로 GLTF 모델 로딩 대기 등 처리. EditorScene은 프로젝트 state를 구독하

여 벽/가구 mesh들을 렌더링합니다. 또한 두 개의 카메라 (탑뷰, 1인칭)를 모두 포함하고 필요에 따라 `makeDefault` 토글. EditorScene 안에서는 모드에 따라 컨트롤 컴포넌트를 조건부 렌더합니다 (예: `mode===`top' ? <OrbitControls ...> : <PointerLockControls ...>`).
- **WallMesh.tsx**: `props.wall` (from, to, thickness, height) 받아 useMemo로 geometry 생성 [1], mesh 구성. 나아가, 문/창 있을 시 그 부분 geometry 잘라내서 (CSG 활용) 재질 적용. 벽 재질: 앞면/뒷면 다른 텍스처도 가능 (도배/외벽).
- **FloorMesh/CeilingMesh.tsx**: 방 polygon 받아 shape -> geometry 생성. 여러 방일 경우 배열 받아 group of floors.
- **FurnitureMesh.tsx**: 개별 가구나 소품. `props.item` (id, type, position, rotation, maybe modelRef). 내부에서 `useGLTF`로 해당 type의 모델을 불러오고 `<primitive object={model.scene} ...>`로 삽입하거나, Drei의 ready-made (Box, Cylinder)로 대체. 성능을 위해 React.memo 적용하여 prop 변경 없으면 재랜더 안 하게 함 [8].
- **PlayerController.tsx**: 1인칭 캐릭터 및 카메라 제어. R3F Rapier의 `<RigidBody>`로 캡슐 생성, children으로 `<PerspectiveCamera makeDefault ref={camRef}/>` 넣어둠. useFrame 훅에서 키보드 입력 체크해 rigidBody velocity 설정, 마우스 delta 받아 rigidBody.rotation 또는 camera rotation 변경. Pointer lock events (onPointerLockChange)로 상태관리.
- **TopViewCamera.tsx**: `<OrthographicCamera makeDefault ref={...}>`를 배치하고, 초기 position/zoom 계산. OrbitControls with target at center. If separate Canvas approach, might not need dynamic toggling, else control enable/disable via mode state.
- **InventoryBar/Sidebar**: React DOM list of items. On click, calls projectStore.addItem or enters first-person placement mode. Possibly uses draggable attributes as described.
- **Navigation**: 상단 Navbar with project name, Save button, mode switch toggle (icon for 3D person vs plan view), help etc. Save button triggers supabase updates or simply indicates "Auto-saved" if real-time.
- **Supabase Client**: encapsulate `createClient` with URL & anon key, maybe wrapper for service key on server if needed.
- **floorPlanAnalysis.ts**: calls AI API. Possibly uses fetch to external endpoint. Might chunk image if needed, handle errors (Claude may give invalid JSON, so code to regex parse as implemented [39]). If GPT-4 Vision via OpenAI, then we might call our own API that relays to OpenAI with proper headers; but likely easier use Anthropich Claude image endpoint.

단계별 구현 순서

개발을 효과적으로 진행하기 위해, 기능을 몇 개의 단계로 나눠 구현할 것을 권장합니다:

1단계: 기본 3D 모델 생성 및 시점 전환 1. **도면 -> 3D 구조**: 우선 **도면 이미지를 수동으로 JSON으로** 변환한 샘플 (혹은 작은 예제 JSON 하드코딩)으로 벽/바닥을 생성해보세요. WallMesh, FloorMesh 컴포넌트 만들고, EditorScene에 렌더링되도록 합니다. 기본 조명/카메라 세팅도 이 단계에서 확인. 2. **시점 전환 기초**: 탑뷰 OrthographicCamera와 1인칭 PerspectiveCamera를 둘 다 씬에 배치하고, 간단한 버튼으로 `mode` state 토글하여 `camera.makeDefault` 변경 또는 separate Canvas show/hide. 아직 이동은 없이, 시점이 바뀌는지만 확인. 천장 가시성 토글도 적용. 3. **Supabase 프로젝트 로드/저장 기본**: Supabase 연동하여 프로젝트 생성 페이지에서 도면 이미지 업로드 -> analyze API 호출 or skip -> dummy JSON 저장. Editor 페이지에서 supabase에서 JSON 불러와 상태 초기화. 일단 AI 없이 수동 JSON으로. 4. **UI 뼈대**: Sidebar에 카테고리 목록 띄우고 InventoryBar도 틀만. Navigation Bar에 Save, Mode toggle 버튼. 기능은 나중에 붙이지만 UI 레이아웃 잡고 Tailwind 적용.

2단계: 1인칭 이동 및 배치 기본 1. **물리엔진 통합**: Rapier 세팅, PlayerController 캡슐 생성. WASD 이동과 점프 구현, PointerLockControls로 시점 회전. 벽과 충돌 확인 (capsule vs wall colliders). 2. **레이캐스트 하이라이트**: 카메라 정면 raycaster 구현해서 부딪히는 Mesh를 감지, 해당 mesh material or OutlineEffect로 하이라이트. 커서 모양 CSS 조작 (e.g., crosshair vs pointer). 3. **기본 가구 배치**: 프로젝트 상태에 furnitureList (e.g., one box table) 하나 넣고 FurnitureMesh 출력. 1인칭에서 인벤토리 1번 누르면 placement mode (or simpler: just press a key to

spawn sample item in front). Ghost preview as semi-transparent Box. Click to place -> add to state. Removal: look at it and press Del -> remove from state. 4. **데이터 동기화**: Supabase DB에 furniture insert/delete on these actions. Another client open same project to verify realtime reflect (if setting up realtime).

3단계: 탑뷰 편집 기능 완성 1. **Drag & Drop**: Implement item drag from sidebar to scene. OnDrop create ghost and finalize. Use snap in drop. 2. **Item selection & move/rotate**: In top view, implement clicking item to select (store selectedId), show bounding box or outline. On mouse down+move on selected, update its pos with snap. On key R, rotate 45deg. Delete key to remove. 3. **Grid & Snap toggles**: Show/hide grid with button. Snap toggle to disable snapping (then items move freely). 4. **UI polish**: Sidebar categories filter items by type. Possibly add search bar to filter list. Ensure mobile responsive (sidebar collapsible).

4단계: 도면 AI 연동 및 정확도 개선 1. **Claude/GPT API integration**: Implement `analyze/route.ts` to call external API with base64 image & prompt. Test with some known floorplan image to see if JSON returns properly. Adjust prompt for better JSON. 2. **Fallbacks**: If LLM fails (no JSON), implement fallback (e.g., try alternate model or show error "분석 실패, 수동 벽 지정 필요"). 3. **OpenCV or CV enhancements** (optional): If certain known limitations (like model often misplacing walls length), consider adding a small OpenCV post-process e.g., if two walls endpoints almost touching but not connected, snap them in JSON, etc. 4. **UI for AI status**: In Create project page, after image upload show "분석 중..." with spinner [67], then proceed. If fail, show error message with option to retry or input scale manually.

5단계: 멀티유저 & collaboration 1. **Realtime presence**: On editor page load, join `project-{id}` channel, track user presence. Show maybe an icon of other users count. 2. **Realtime updates**: Subscribe to `postgres_changes` on furniture table for this project. On insert, if not self, add to state. On update (position/rotation), update state. On delete, remove. Already supabase handles uniqueness by primary keys. 3. **Avatar rendering**: For each presence user (excluding self), create an avatar (simple colored capsule or name tag). Update its position as that user shares (they can broadcast their camera position every X ms via supabase channel). 4. **Editing conflict**: Basic rule can be last write wins. To avoid jitter if two people drag same item, perhaps lock item on select: when user selects an item for move, broadcast "lock item X by user Y". Others mark it as locked (grey out). If still both do, oh well, keep last.

6단계: Visual polish & optimization 1. **Lighting & effects**: Add environment map lighting (e.g., Drei `<Environment preset="city" />` or custom HDRI). Tweak Hemisphere and directional lights inside. Add shadows (enable `shadows` on Canvas [68], set light shadow props). Test performance. Add postprocessing: subtle bloom, SSAO if performance allows (SAO pass). 2. **Material details**: Replace wall/floor plain colors with textures from Polyhaven for floor, a default wall paint, etc. Could let user choose in UI (tile category). 3. **Model library**: Integrate actual furniture models for more variety. Optimize by preloading or use draco compression. Possibly integrate gltf-jsx to convert some to code if simpler ones. 4. **Performance**: Check Lighthouse. Optimize: Use `<Suspense fallback={<LoadingCanvas />}>` to show a loading screen while heavy models load. Use `React.Suspense` for non-blocking data fetch. Add code-splitting for routes not in use. Memleak check with React devtools (particularly check if removing items frees memory). 5. **UI/UX improvements**: Add undo/redo for placements (store a history stack). Add confirmation modals for destructive actions. Implement help modal explaining controls (WASD, drag controls, etc). 6. **Dark mode & theme**: If needed, Tailwind dark mode classes or toggle to suit user preference [65].

이러한 단계를 거치면, 프로젝트는 점진적으로 완성됩니다. 각 단계 끝마다 **실행 가능한 상태**가 되어야 하므로, 우선순위가 높은 3D 생성/편집 기능을 1~3단계에서 모두 구현하고, 4단계 AI, 5단계 멀티유저, 6단계 품질개선은 추가 기능으로 배포 후에도 지속 개선할 수 있습니다. 실제 내부 가이드에서도 AI 도면 분석 0%→100%, 1인칭 카메라 0%→100% 등 진행률 개념을 제시했는데 [41], 이를 우리도 유사하게 관리합니다.

결론 및 기대 효과

이 Zero-to-One 개발 가이드를 따르면, 기존 대비 **시각적 완성도**와 **도면 인식 정확도**를 크게 끌어올릴 수 있습니다. 구조적으로도 클린한 폴더 구조와 문서화로 유지보수가 쉬워지고, Supabase 백엔드와의 긴밀한 통합으로 **협업과 확장**이 용이한 플랫폼이 될 것입니다 [69]. 특히: - 리팩토링 중심 접근 대신 **근본적 신규 구축**을 통해 성능(FPS 60)과 안정성 향상 [23], - React Three Fiber 및 최신 생태계 도구 도입으로 개발 효율과 UI 품질 동시 제고, - AI+CV 혼합 전략으로 자동 도면 3D 변환 정확도 개선 및 추후 데이터 축적으로 **학습 고도화** 가능, - 1인칭 & 탑뷰 듀얼 모드 제공으로 사용자에게 차별화된 UX, 경쟁 서비스 대비 우위 확보, - 실시간 멀티플레이/공유 기능으로 커뮤니티 활성화 및 사용자 참여 증진.

마지막으로, 프로젝트를 진행하며 항상 **사용자 체감**을 최우선으로 기능을 다듬어야 합니다. 시각적으로 부드럽고 아름다운 렌더링, 정확하게 구현된 평면 구조, 편리한 편집 UI가 결합될 때 비로소 사용자는 "와, 내가 평면도만 올렸는데 바로 이런 3D 공간이!"라는 감동을 얻을 것입니다. 이를 위해 위에서 제시한 기술 요소들을 단계별로 착실히 구현하고, 필요시 테스트와 피드백을 반영하면서 조정하면 될 것입니다.

以上을 종합한 이 가이드는 **Plan2Space** 프로젝트를 성공적인 100점짜리 플랫폼으로 이끌기 위한 청사진입니다 [70]. 앞으로 구현 과정에서 예기치 못한 문제가 생기더라도, 이 전체 로드맵을 참고하여 균형 잡힌 해법을 찾아나간다면 궁극적으로 목표한 수준의 제품 완성을 이룰 수 있을 것입니다. **행운을 빕니다!**

참고 자료: 내부 설계 문서 및 코드 조각을 필요 시 참조하여 구현 세부를 확인하시기 바랍니다 (예: AI_FLOOR_PLAN_GUIDE.md, CODE_TEMPLATES.md 등). 또한 관련 라이브러리 공식 문서 및 예제 (React Three Fiber docs, Supabase docs 등)도 적극 활용해 주세요.

---

[1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14] [15] [16] [17] [18] [19] [20] [21] [22] [23] [24] [33] [34] [35] [36] [37] [38] [39] [40] [41] [42] [43] [44] [45] [46] [47] [48] [49] [50] [51] [52] [53] [54] [55] [56] [57] [58] [59] [60] [61] [62] [63] [64] [65] [66] [67] [68] [69] [70] working.pdf
file://file_0000000013a0720b905ffd44ed1af5e7

[25] What are the various techniques for detecting walls in buildings ...
https://dsp.stackexchange.com/questions/16465/what-are-the-various-techniques-for-detecting-walls-in-buildings-architectural

[26] [27] Find contours for walls in floor plan images - Python - OpenCV Forum
https://forum.opencv.org/t/find-contours-for-walls-in-floor-plan-images/11640

[28] [29] [30] [31] [32] Floor Plan Analysis with Computer Vision: Try It Free
https://blog.roboflow.com/floor-plan-analysis-computer-vision/